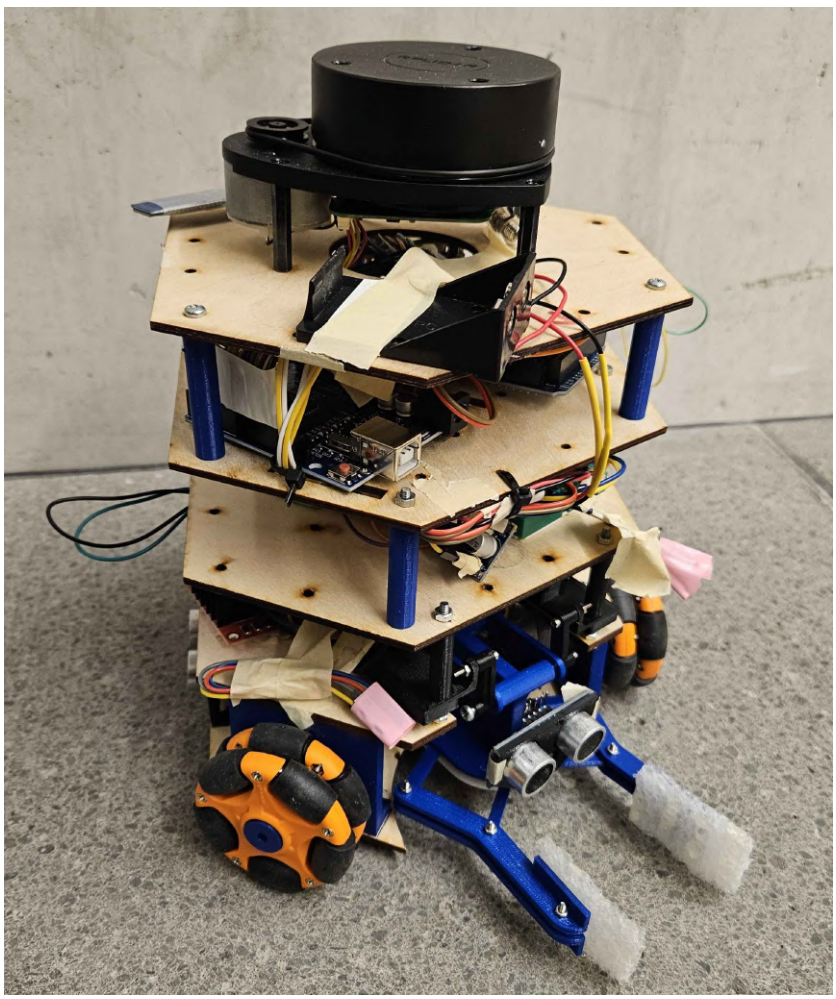
MIE444 Project Report

Team 9

Justin Narayan, 1008062784
Trent Rossos, 1008073914
Claire Zhang, 1008001822
Alice Liu, 1008092369

December 3rd 2025



Contents

1 Executive Summary **4**

2 Detailed Rover Control Strategy **5**
2.1 Obstacle Avoidance . . . 5
2.1.1 Electrical . . . 5
2.1.2 Mechanical . . . 5
2.1.3 Programming . . . 6
2.2 Localization and Navigation . . . 7
2.2.1 Electrical . . . 7
2.2.2 Mechanical . . . 7
2.2.3 Programming . . . 8
2.3 Block Pickup and Delivery . . . 9
2.3.1 Electrical . . . 10
2.3.2 Mechanical . . . 10
2.3.3 Programming . . . 10
2.4 Integration . . . 11
2.4.1 Electrical . . . 12
2.4.2 Mechanical . . . 12
2.4.3 Programming . . . 12

3 Final Results **15**
3.1 Obstacle Avoidance . . . 15
3.2 Localization and Navigation . . . 15
3.3 Block Pickup and Delivery . . . 16
3.4 Integration . . . 16

4 Discussion **17**
4.1 Best Features . . . 17
4.1.1 Gripper . . . 17
4.1.2 Drivetrain and Chassis . . . 17
4.1.3 Modular Programming Architecture . . . 18
4.2 Worst Features . . . 19
4.2.1 Power Management . . . 19
4.3 Alternative Designs . . . 19
4.3.1 H-Drive . . . 19
4.3.2 Autonomous State Management . . . 20

A Electrical **22**
A.1 Initial Proposal Design . . . 22
A.2 Milestone 1 . . . 23
A.3 Milestone 2 . . . 24
A.4 Milestone 3 . . . 25
A.5 Individual Wiring Diagram . . . 26

A.6 Power Distribution . . . 27

B Mechanical **28**
B.1 Drivetrain and Chassis . . . 28
B.2 Gripper Assembly, CAD . . . 29
B.3 Wrist Subsystem, CAD . . . 29
B.4 Arm Subsystem, CAD . . . 30
B.5 Gripper Assembly, Final . . . 30
B.6 Full Assembly, CAD . . . 32

C Code Excerpts **34**
C.1 Communications Interface . . . 34
C.1.1 Translate Interface, Milestone 1 . . . 34
C.1.2 Translate Interface, Final . . . 35
C.1.3 Translate Interface, Python . . . 37
C.2 Control Logic . . . 40
C.2.1 Drivetrain Logic, Milestone 1 . . . 40
C.2.2 Forward and Inverse Kinematics . . . 41
C.2.3 PID Control Implementation . . . 42
C.3 Architecture . . . 46
C.3.1 Initial State Diagram . . . 46
C.3.2 Hierarchical Structure . . . 47
C.3.3 Taskmaster Operational Loop . . . 48
C.4 Localization . . . 49
C.4.1 Particle Filter Weight Update . . . 49
C.4.2 Inverse Kinematics . . . 50
C.4.3 LiDAR Visualization . . . 51

D Kiwi Drive Mathematics **52**
D.1 Movement Diagrams . . . 52
D.2 Kinematics . . . 53
D.3 Encoder Calibration Procedure . . . 54

1 Executive Summary

This report outlines our team's process of designing, constructing, and testing an autonomous rover. Our rover was required to complete three fundamental tasks within five minutes, broken into the following milestones: (M1) avoiding obstacles while navigating through a walled maze of 1' by 1' squares with a known map, (M2) localizing itself from an arbitrary initial state and pathfinding to preset locations, and (M3) locating, retrieving and delivering a small block to a specified dropoff point. In addition, the final milestone also assessed our ability to integrate all these subsystems to achieve a successful unassisted run. Additional points were awarded for aesthetics, speed, and more automated completion of all tasks.

Our team designed our rover to operate primarily under manual control, and many key design decisions evolved from this focus. Our selection of a physically small robot diameter and Kiwi drivetrain allowed for unconstrained movement in any direction while minimizing the rover's footprint, making navigation around tight corners more comfortable. Our integration of a LiDAR unit mounted atop our robot allowed for frequent 360° scans of our environment, accelerating convergence of our localization algorithm and providing ample information for manual pathfinding and obstacle avoidance. Our gripper incorporated a sleek, retractable arm with a simple yet effective gripper to grasp the block firmly while minimizing the impact on drivetrain handling and the chassis size. Additionally, our use of both an Arduino Uno and Mega allowed careful delegation of our robot's functionality: one microcontroller focused on low-level drivetrain controls, while another prioritized fast Bluetooth communication with the human controller and on-demand sensor readings.

At the time of submission, our robot is able to successfully achieve all of the above functionality. However, due to our manual control interface and imprecise manufacturing tolerances and calibration of our electronics, our rover is fairly slow and cannot consistently complete all tasks within the allotted five minutes. Additionally, we relied on occasional human intervention, such as the manual placement of the block once in the loading zone (in lieu of the rover locating the block itself) and minor adjustments to the robot's position to aid obstacle avoidance, which incurred penalties during our runs. Finally, our rover is subject to unreliable electronics, where our Bluetooth, LiDAR, and microcontroller occasionally brown out causing our rover to fail which requires manual intervention to power cycle and reconnect to the control interface. As a result, despite the proven functionality of each individual subsystem on our robot, the success of our robot's integration as of the final milestone is limited. Our primary next steps to improve the performance of our robot include (1) integration of ground-level ultrasonic sensors to locate and navigate to the block, (2) more precise calibration and tuning of hardware and electronics for more consistent drivetrain control, and (3) automation of pathfinding to increase the rover's speed and more reliably avoid obstacles.

For videos of our robot's performance across all three milestones, as well as recordings of our computer control interface and our codebase, please see this Google Drive folder. Our code is also available on Github.

2 Detailed Rover Control Strategy

2.1 Obstacle Avoidance

For M1, the rover had to manually navigate the maze with wireless, manual control under direct human line-of-sight (no sensors required). The rover must drive, turn, and avoid obstacles (including walls and corners), with points awarded based on the speed to navigate a certain number of squares.

2.1.1 Electrical

As shown in Appendix A1, we used the Arduino Uno and Mega board (UNO, MEGA) with a HC-05 Bluetooth Module to control three 6V DC motors (used for Kiwi Drive strategy, explained in the section 2.1.2). The Bluetooth was powered via a LM2596 voltage converter, stepping the primary 12V battery down to 5V. The three motors use two L298N motor drivers connected to PWM and regular GPIO pins on the Uno; note that each motor had attached encoders which were not used in M1. Both the Uno and motor drivers are powered by an additional LM2596 voltage converter down to 9V from the 12V battery.

From the Uno datasheet [8], 7-12V is the recommended V_{in} so we chose to send 9V to V_{in} (Appendix A.2). However, in the final milestone, we power both boards with 12V directly from the battery after repurposing the LM2596 for a different subsystem (Appendix A.4).

From the L298N datasheet [4], the V_{in} is rated for 5-35V and since we are powering 6V motors, we chose to send 9V to V_{in} after physically testing the drivers and noticing significant voltage drop across the L298N H-bridge circuitry. This voltage drop is expected through the transistors, as stated in the datasheet [4]. The driver is expected to lose up to $\sim$3.2V at 1A of current, or up to $\sim$4.9V at 2A, and if the driver is older or previously overheated, the voltage drop may be worsened due to higher internal resistance and less efficient conduction after excessive heating. Thus, even when providing the maximum motor speed input to the L298N from the Uno's PWM pins, we were confident we would not damage the 6V motors because the drivers would send approximately 5V to the motors rather than a full 9V. We also probed the output terminals of the drivers while sending drive commands to verify that the actual voltage the motors were receiving is safely under 6V.

The wiring of individual Bluetooth (BT), motors (M1/2/3), motor drivers (MD1/2) is shown in Appendix A.5

2.1.2 Mechanical

The rover uses a three-wheel Kiwi drive powered by 6V DC motors with omni wheels, enabling holonomic motion (Appendix B.1). Omni wheels can slip freely, allowing independent control of all navigational degrees of freedom - the 2D position in the maze (denoted as x, y), and orientation (denoted as θ). Kiwi drive enables rotation and movement in any direction, supporting point turns, translation, strafing, and movement between arbitrary states of (x, y, θ) (Appendix D.1). This holonomic drivetrain simplified path planning and allowed confident, nimble control in the confined environment of the maze.

During pre-M1 motor and wheel testing, we observed significant differences in rotational speed and slip, owing to poor manufacturing tolerances, and required additional software tuning. Two motors had similar speeds and were selected as the primary driving motors to reduce software tuning complexity. The third motor rotated slower and slipped more on the maze surface, so was mainly used for point turns since we only implemented translation and rotation functionality.

To minimize the rover's footprint, we used a stacked structural system of laser-cut 3mm plywood plates. Each layer served as a mounting platform for electrical and mechanical components, increasing vertical surface area without enlarging the horizontal footprint. Cutouts in the plates allowed internal routing of wiring.

2.1.3 Programming

We created a Github repository early on and used PlatformIO to set up our workspace. Based on previous experience, PlatformIO allows for seamless development across several different microcontrollers, which proved useful as we always intended to use several Arduinos. All Arduino development was done in C++ (as our team had prior firmware experience), and all code on the receiver laptop was written in Python. For M1, we primarily used an Arduino Uno to control and process all drivetrain commands. The Uno communicated via UART with an Arduino Mega, which would forward all received messages between the Uno and the Bluetooth module connected to the human operator's computer. Though somewhat redundant for this milestone (since the Mega was just a middleman), we knew this architecture would be essential for future milestones and so prototyped it here. As Arduinos are very limited on flash memory, we limited the number of imported libraries (such as the C++ standard library) and opted to rewrite required utilities such as `memcpy` and `strlen`.

Following good firmware engineering practices, we opted for a modular, abstracted firmware architecture – this allows for easy addition of new functionality, and helps stratify functionality to simplify debugging by isolating high-level control code and low-level drivers (see Section 4.1.3). We constructed an abstracted communications wrapper for Arduino's low-level Serial interface, allowing for easy hot-swapping between Bluetooth communications during testing and USB communications during debugging. This wrapper implemented a queue to continuously poll the Serial interface and parcel the raw bytes into encoded messages, ensuring no incoming data was lost if there was a backlog of commands. For M1, our encoding protocol consisted of a set of known strings (such as "w", "a", "s", "d" for longitudinal and rotation commands) followed by an encoding token ("$"), since all messages were either drivetrain commands or acknowledgments upon command receipt (Appendix C.1.1). This simple protocol was very effective and laid a strong baseline to expand upon for more complex communications in future milestones.

Drivetrain controls were similarly straightforward, as our team had previously written firmware drivers for the rover's L298N motor controllers, with customizable speed input via the PWM signal to the driver's enable pin. Empirically, we observed significant differences in our motor's output torque provided the same speed command. For Kiwi drive, parity of motors is essential as any deviation or slip will cause significant drift in the robot's position and orientation over even short movements. Fortunately, we controlled our robot with line-

of-sight for M1, allowing quick recognition and adjustment of path drift with a quick rotation command. However, we still prioritized tuning effective constant speed gains for the motors to achieve approximately straight translation and perfect pivot rotation to reduce the burden on the human operator. Although the discrepancies between motors were not fully linear (i.e. the percent difference in motor speed varied from a 50% speed command to a 100% speed command), we had success with this approach. The module controlling the incoming drive commands and drivetrain interface also implemented an automatic stop, where all motors would brake if no command had been provided in the last 100 milliseconds – this further reduced the burden of the operator to issue a manual halt command (Appendix C.2.1). Even with Bluetooth communications, we observed minimal delay with commands to our drivetrain interface, and we were content with our rover's responsiveness throughout all successive milestones.

2.2 Localization and Navigation

For M2, the robot had to localize in the maze from a random starting position and orientation and navigate to a desired end location. As with M1, the robot had to avoid obstacles, but the robot now had to be controlled without line-of-sight, requiring manual or automated controllers to make use only of data from sensors mounted on the robot.

2.2.1 Electrical

As shown in Appendix A.3, we added the Arduino Mega board, motor encoders, and LiDAR to the electrical setup used in M1 (Appendix A.2) to accommodate for the additional objectives of M2. As M2 prohibited line-of-sight control, we opted to attach the encoders to provide feedback as the motors actuated to monitor the rover's motion. After discussion with the Professor, we decided to attach the encoders to a separate board than the rest of the motor controls, as they operated on interrupts and could disrupt other operations on the microcontroller as the robot was moving. We brought all motor controls to the Mega after verifying functionality of the required PWM and GPIO pins, and isolated the encoders to the Uno. Both boards communicated via UART, with Bluetooth and LiDAR communications handled by the Mega, making use of its additional Serial communication ports. The Uno handled only the encoder interrupts and basic communication of encoder readings upon request from the Mega (see Section 2.2.3).

We kept one voltage converter (12V battery to 9V) to power the Mega, Uno, and two motor drivers. We added the LiDAR and three encoders to the 5V rail that previously was only powering the Bluetooth from a second voltage converter (12V to 5V). The wiring of individual LiDAR (LIDAR) and motor encoders (ME1/2/3) is shown in Appendix A.5, with respect to component power rating stated on datasheets [6][9].

2.2.2 Mechanical

No major modifications to the mechanical components of the rover were made for M2. An additional mounting plate was added following the modifications and addition of electrical

components. Additional mounting space on the rover also allowed for preparations for M3 by clearing the space allotted for the gripper system.

2.2.3 Programming

For M2, sweeping changes were made to the microcontroller architecture to incorporate all additional sensors and control logic for localization. The Mega's functionality was expanded to forward messages to and from the Uno as well as run vital rover subsystems. Abstract `Controller` classes were created implementing a customizable `MessageQueue` for their incoming and outgoing messages using a custom `RingBuffer` interface. This structure allowed for explicit isolation of each controller's functionality and communication interfaces, leading to easier and more targeted debugging of the rover's functionality. Additionally, each microcontroller implemented a `Taskmaster` class which would control an incoming communications interface (Bluetooth for the Mega, and the UART port for the Uno) and all `Controller`s on the board (Appendix C.3.2). The `Taskmaster` executed a simple loop of (1) polling the communications interface for received messages, (2) delivering an incoming message to any `Controller` instances expecting to receive that message type, (3) allowing each `Controller` to execute a process loop to run its required operations, and (4) retrieving any outgoing messages from each `Controller` and sending them on the communications port (Appendix C.3.3). This hierarchical interface proved to be extremely effective and made the addition of new controllers such as the LiDAR or encoders very simple.

Our Message interface also received large updates. Previously, all messages were for the drivetrain and were easily decoded with an enum lookup table (Appendix C.1.1). With the rover's new functionality, we updated our encoding protocol to begin each message with a byte encoding the type of message (to allow a `Controller` to efficiently identify which messages they receive and send), permitting up to 255 distinct types of messages. Additionally, as the LiDAR and encoders would be sending struct information, it was recognized that any of their raw bytes (i.e. corresponding to a distance reading) could match the tokenization character, resulting in the receiver prematurely clipping the message. Thus, a second encoding byte for the size of the following message content was added. Our `Translate` firmware model implemented a generalized interface for serializing and deserializing all enums and structs into `Message` objects and vice versa, avoiding the manual creation of lookup tables which was tedious and prone to human typos (Appendix C.1.2). This was an onerous process, but the Translate interface was extremely robust, and its behaviour was easily replicable on the Python receiver (Appendix C.1.3).

Using a premade Arduino library for the RPLidar module [5], we built a controller which could receive ping requests for the LiDAR from the computer and send back data for a complete 360° scan. Each LiDAR data point included an angle (relative to the robot's heading) and a recorded distance. As well, each LiDAR request would also ping the encoders, allowing for computation of the exact change in the robot's state (x, y, θ) via inverse kinematics in our localization process (Appendix D.2). As with our motors, the encoders had to be carefully tuned to ensure their reported data (the distance rolled by the corresponding wheel along the ground since the robot turned on) was accurate. The tuning process involved manually commanding careful 360° point turns of the robot and adjusting the gains until each encoder

reported the expected distance for a complete rotation (Appendix D.3).

As our LiDAR provided such dense information on each scan, we opted to implement a Monte Carlo particle filter for localization to make optimal use of our sensors. In a particle filter, we initiate thousands of particles in uniformly random positions and orientations across our known map, and procedurally refine our distribution of particles by moving particles based on the applied drivetrain commands (the "control input") to the robot, and prioritizing particles whose simulated sensor readings match our rover's data. On every received LiDAR scan, we would compute the change in the robot's state based on the previous and current encoder readings (Appendix C.4.2), and then compare the received LiDAR data to a set of reference LiDAR readings for particles in the known maze. However, we noticed significant discrepancies between the absolute distance values of the provided reference LiDAR data on Quercus and our empirical data – the reference data was consistently a few inches less than our observed data. These differences made it very challenging to map the angles of our empirical data (in the robot's frame, where $0°$ is "forward") to the reference data (in the global frame, where $0°$ is "right"), and our estimate ultimately failed to converge as our computed LiDAR similarity measures were always extremely low.

Though all of the firmware worked successfully for M2, the issues with our Python localization procedure prevented us from showcasing this in our M2 run. Due to time constraints from our localization issues, we were unable to build an adequate remote control interface to navigate the robot without line-of-sight for M2, and we also did not have time to integrate or validate any functionality with the simulator. We had no automated control or pathfinding interface implemented, so our intended approach was a manual navigation and obstacle avoidance "algorithm" making use of our localization estimate and the human operator's knowledge of the map layout. Without time to create a LiDAR visualization for the human operator, we relied on a haphazard text printout of the raw LiDAR angle and distance measurements, which proved challenging to interpret while operating the robot.

As well, issues with our power management caused intermittent brownouts of the rover when pinging the LiDAR, which manifested with noisy data or zero bytes being sent on the Bluetooth connection before the rover abruptly shut off – at one point, the TX pin on our Bluetooth even became defective. We noticed these issues did not happen when using a physical USB connection in lieu of Bluetooth, but we were unable to resolve these issues adequately in time for M2. As a result, our robot collided with walls, required several manual power cycles, and was unable to complete the milestone's objectives.

2.3 Block Pickup and Delivery

For M3 the rover must now drive to a known loading zone, find the load (a 1' by 1' block), pick it up and lift it above the ground, drive to a desired end position, and place the load. all controlled without a direct line of vision to the robot. As with M2, the robot is controlled without line-of-sight, with points awarded for speed and automation, and penalties incurred for manual intervention or simplification (such as placing the block in a known position in the loading zone).

2.3.1 Electrical

As shown in Appendix A.4, we augmented the existing M2 functionality, by adding 2 servos for the block pickup mechanism (or "gripper"). We also updated the power distribution of components receiving 5V to reduce current draw on the single 5V rail on our breadboard (discussed further in Section 4.2.1). A separate 9V battery stepped down to 5V was added to power the Bluetooth, reducing connection failures to voltage fluctuations on the rover and ensuring continued connection even when the microcontroller is off. Additional voltage converters stepping the 12V battery down to 5V were also added to isolate the power supply for the servos (S1/2), the LiDAR, and the motor drivers (see Appendix A.5). Due to time constraints for this milestone, we did not add the ultrasonics or additional sensors outlined in our initial design to aid with block detection (Appendix A.1).

2.3.2 Mechanical

The block pick-up mechanism (or "gripper") underwent multiple design iterations to fit the rover's compact size. Initial concepts employed larger stepper and MG996 servos but were replaced with MG90S micro servos to ensure proper fit. Stall torques from the MG90S datasheet [2] were verified against the estimated block mass and resulting moment arm during the block pickup and delivery procedure, confirming sufficient torque for grasping and lifting.

The gripper has two components: the wrist, which clamps the block, and the arm, which lifts it vertically (Appendix B.2). The wrist uses a symmetric crank-rocker linkage, mechanically synchronized via spur gears, and driven by a micro servo (see Appendix B.3). This configuration ensures simultaneous, symmetric jaw motion, reduces actuator count, and enables fast cycles. The arm, mounted to the second structural plate, is actuated by a second servo (see Appendix B.4); its rotation lifts the wrist via a parallelogram linkage while keeping it parallel with the ground, evenly distributing the block's weight along the contact surface.

During M3 testing, the PLA components occasionally slipped, causing block drops - the servos were weaker than expected. This issue was mitigated by adding styrofoam padding at gripper-block interfaces, increasing contact area and friction to improve reliability. Gripper integration is further discussed in Section 2.4.2.

2.3.3 Programming

After our issues in M2, we rewrote all of the Monte Carlo localization Python code from scratch. In particular, we changed the `update_weight` function (which compared real and simulated LiDAR data), to measure the similarity of normalized LiDAR readings (between 0 and 1) rather than absolute readings (Appendix C.4.1). This adequately corrected for the observed discrepancies, and our rover was able to efficiently localize from anywhere in the maze. This approach meant that the rover would see no difference between being centered in a 1' by 1' square versus a 2' by 2' square, but in the known map, there were no locations where this ambiguity would cripple the localization process in our testing.

We spent significant time refining parameters (such as standard deviations for measure-

ment and control error) to optimize the speed and accuracy of our localization. In particular, we noticed significant drift in our encoder readings over time even after careful calibration, likely due to inconsistent slip behaviour of the cheap wheels and motors, which would lead to our estimated position visually "lagging" behind our real position if our robot moved too much between steps. Our best approach to account for this was significantly increasing the estimated error on our control input (based on our encoder readings) and moving only in short bursts, which required significantly longer to successfully localize.

As with M2, we opted for the operator to manually pathfind rather than implementing any true algorithms for navigation or obstacle avoidance. We created a simple LiDAR visualization which updated upon every localization step, allowing for straightforward and confident navigation by the human operator (Appendix C.4.3). As with M1 and M2, we did not use the provided simulator whatsoever, and all design, testing, and debugging was done completely with the physical robot in the maze, representative of the actual setup for M3. Though we had time to integrate simulator functionality, we recognized implementing our control and localization logic would still be a significant undertaking, and this simply did not seem worthwhile once we had settled on a successful localization and navigation procedure.

With our updated power management, we generally resolved our previous fatal brownout issues. However, LiDAR scans still occasionally caused rover shutdowns, requiring manual interventions during our M3 runs and inducing a reset of our localization progress. The same symptoms of zero packets flooding the Bluetooth interface before rover failure occurred even after swapping Bluetooth modules, but never when using a physical Serial connection. As the firmware uses an abstracted communications interface treating these two methods identically, this indicated the problem was a deeper issue with our electronics architecture.

Though little functionality was explicitly added for our first M3 attempt, some programming architecture was reworked in preparation for block pickup and integration tasks. As the Mega would eventually have to control the gripper due to its surplus of PWM pins, we opted to move all drivetrain controls back to the Uno to avoid running out of flash memory on the Mega (see also Section 4.1.3). We began implementing an automated control interface, where the drivetrain would be commanded to move a certain $(dx, dy, d\theta)$, and could compute the required changes in encoder readings via inverse kinematics (Appendix C.2.2). As the control logic would be running at a very high frequency ($\sim$50Hz), it was optimal for the Uno to manage the drivetrain logic as it had direct access to the encoders – on the Mega, there would be a noticeable delay when receiving encoder values from the Uno over UART. Though automation was not complete at this stage, this process involved building out the Uno and Mega's communication interface (including an `EnvoyController` for the Mega to send requests to the Uno without explicit command from the user) and cross-referencing the Uno's low-level kinematics math to ensure it matched the conventions used in the Python localization logic.

2.4 Integration

For our rover's final trials in this project, the requirements are exactly the same as M3. The focus in this section is discussing our procedure of testing and debugging our fully

integrated system for our last attempts at M3.

2.4.1 Electrical

Appendices A.2, A.3, A.4 shows the integration of more components needed for each milestone. Appendix A.2 contains the drive train (Motors M1/2/3, drivers MD 1/2), the Bluetooth Module (BT), Uno and Mega. Appendix A.3 adds the motor encoders (ME1/2/3) and LiDAR. Appendix A.4 adds the servos for the arm mechanism (S1/2). As more components were added, we also had major wiring changes to isolate the Bluetooth and LiDAR and fix our power management with a new breadboard, voltage converters/regulators and a 9V battery (discussed further in Section 4.2.1). All changes to our electrical setup throughout all milestones were done after validating each component's voltage and current ratings from their respective datasheets [7, 8, 3, 1, 6, 9, 2].

2.4.2 Mechanical

While the mechanical elements remained largely unchanged for the final runs, we focused on tidier assembly to reduce footprint, improve ease of assembly/disassembly, and enhance reliability and sturdiness. The drivetrain and gripper were designed concurrently with the electronics layout to ensure spatial compatibility. A vertically stacked chassis with multiple mounting plates allowed drive motors and gripper to be integrated without largely increasing the footprint. These mechanisms were placed at the bottom plates to maintain a low center of mass and balance heavier electronics like the LiDAR which was placed at the highest plate for optimal visibility.

Integrating the gripper required key mechanical changes due to the placement of electronics and space needed for full actuation. The wrist has a wide opening to tolerate block positioning. An ultrasonic sensor at the gripper base was intended to verify block acquisition but was not used in the final runs. While the wrist was designed to clear the DC motor mounts, screw heads occasionally caught on the base, creating issues with the mechanism jamming. This was mitigated with cutouts in the base plywood and raising the gripper using hex nut spacers (Appendix B.5). Larger cutouts would increase actuation range, but was minimized to maintain stability for the drive train on the same layer.

Integration of the arm revealed limitations that could not be mechanically resolved. DC motor encoders and wiring behind the gripper prevented full arm retraction, limiting lifting range block transport. Software procedures accounted for these spatial changes and prevented over-actuation of the servos into the chassis, which could overwork the servos or impair control of the block.

2.4.3 Programming

An excessive amount of time was spent tuning the automated control interface for the final M3 runs. A Proportional-Integral-Derivative (PID) controller for arbitrary movement commands $(dx, dy, d\theta)$ was developed and calibrated, exploring both control directly of the $(dx, dy, d\theta)$ terms as well as the raw encoder and motor values (Appendix C.2.3). Though

the PID control interface worked successfully, tuning the exact gains proved extremely difficult given the unreliability of the hardware. We previously mitigated the difference in motor characteristics by running all wheels at constant, rapid speeds while translating or rotating. However, for arbitrary commands, the motors must move at variable speeds, causing extremely fragile behaviour due to the motors' enormous deadbands (up to 25% of the max PWM signal) and inducing unpredictable slip in the drivetrain. Motor behaviour also varied based on the current charge of the battery as well as the floor material beneath the drivetrain, leading to further complications while tuning PID parameters.

Our underlying kinematics control logic was completely successful, accurately getting the rover to the desired final state, and avoiding brownouts of the motor driver H-bridges by strategically avoiding sporadic changes in motor direction. However, the variability in the drivetrain hardware led to inconsistent pathing, often steering the robot into walls if commands were executed in the tight corridors of the maze. Discussing with other teams near the M3 deadline, we realized most groups with our drivetrain setup had opted to swap wheels or motors to mitigate these issues. Due to insufficient time to replace our faulty hardware, we unfortunately opted to scrap our automation interface altogether, ultimately wasting a significant amount of time and programming effort.

The integration of the gripper was far more successful, largely due to the utility of our modular firmware architecture (see Section 4.1.3). By developing low-level servo motor drivers, a gripper command and response interface, and a gripper `Controller` to run on the Mega, we were very quickly able to integrate this functionality with our robot with minimal programming complications – our biggest issue was tuning the servo target positions based on the chassis geometry.

With the gripper installed, we hoped to implement block detection functionality using our mounted ground-level ultrasonics. We successfully programmed an `UltrasonicController` which, when requested by the human operator, would execute a controlled 360° point turn, pinging two ultrasonic sensors at $\sim$10 degree increments for their observed distances, and reporting the data back along with encoder readings for each iteration. On the Python receiver, we could use inverse kinematics to compute the exact position of each ultrasonic reading in the robot's final frame, and superimpose the results on our LiDAR visualization. Our "algorithm" was effectively using the ultrasonics to perform a "ground-level LiDAR scan", which would then allow for the manual identification of anomalies between both sensors in the visualization. As the only obstacles in the maze are the block (at ground-level) and the walls (at and above ground-level), we could be certain that any detected anomaly would necessarily be the block, allowing us to drive up to it and grasp it, with human assistance if necessary.

Unfortunately, we experienced familiar brownout issues when pinging the ultrasonic sensors, even though our LiDAR behaviour was much more stable. All Serial communications over Bluetooth and between both microcontrollers so far used a baud rate of 57600. As we frequently observed communications failures accompanying our brownouts, we suspected complications arising from this communication throughput and so reduced all baud rates to a minimal 9600. Though our rover still had full functionality, we still experienced sporadic power management failures, worsened by repeated pings of ultrasonic sensors. Prioritizing

power stability in our final M3 runs to ensure we made it through the maze, we scrapped our block detection efforts and instead opted to manually place the block in our gripper when at the loading zone.

Our intended functionality for our final integration runs was equivalent to M3, with the addition of block pickup and delivery. Unfortunately, a significant amount of programming work was completed successfully but went entirely unused due to faulty hardware and inconsistent electronics, constituting a significant waste of time and human resources.

3 Final Results

3.1 Obstacle Avoidance

Our robot performed perfectly during M1, successfully navigating 20 blocks while avoiding obstacles under manual control. The rover reliably received Bluetooth commands, drove forwards and backwards, rotated about a point, and automatically halted when receiving no drive commands. Our mechanical philosophy of minimizing the robot's footprint while accommodating all components with vertical stacking proved highly beneficial, giving us extra clearance to maneuver freely and take tighter corners at higher speeds without risk of collision.

In future milestones, our robot's obstacle avoidance performance suffered somewhat. Our hardware components (including motors, motor controllers, and encoders) degraded over the course of the project, requiring several cycles of exchanging components and recalibrating our drivetrain parameters. Although the LiDAR provided sufficient data to navigate without line-of-sight, the drivetrain components themselves were unreliable, and tuning drivetrain parameters to drive straight was a constant challenge. As a result of this faulty hardware, our robot occasionally collided with walls, requiring manual intervention to reposition it.

3.2 Localization and Navigation

During M2, we added the LiDAR to use as our primary source of information for localization. Our procedure for localization involved the manual controller repeatedly (1) moving the robot, (2) pinging the LiDAR for a full 360° scan and sending this information to the Python receiver, where (3) a localization script would compare the constructed LiDAR reading to a set of reference LiDAR readings for each position in the maze, and (4) step the particle filter (see Section 2.2.3).

Though this procedure was generally successful, we encountered frequent crippling power failures with the LiDAR and Bluetooth modules intermittently browning out, causing communication dropouts and complete system failure (see Section 4.2.1). This required manual intervention to power cycle the robot and reconnect Bluetooth several times per run, resulting in failure to complete the milestone. Though our localization strategy was sound and was accurately able to identify the rover's position and orientation within a short time, we were unable to successfully showcase this as each power cycling of the robot reinitialized our localization procedure.

For future milestones, the power instability was broadly remedied, and we successfully localized after a few iterations of small movement inputs and LiDAR pings, navigating manually with our LiDAR visualization. Our localization parameters, including modeled error for our control inputs, were not optimal however, as we observed our robot's localized position would often drift if the robot collided with walls or moved a large amount between localization steps. Our best solution was to move slowly in short bursts or periodically reset our localization if the human operator noticed a mismatch between the LiDAR readings and localization

output, so our performance on this metric could certainly have been improved.

3.3 Block Pickup and Delivery

During M3, the mechanical arm reliably grasped the block, lifted it with ∼2mm clearance, and retracted back after loading. Once at the dropoff point, we were able to quickly and reliably deposit the block. The compact, retractable arm minimally increased the robot's footprint, enabling easy navigation to the drop-off point while lifting the block. This design complemented our control strategy, which prioritized forward translation; incorporating strafing or other non-longitudinal motion would complicate navigation due to the robot's non-uniform radius.

Time constraints prevented implementation of block location or pickup verification. Although ground-level sensors were mounted and a 360° scan routine was tested, they were not integrated due to power management issues and limited time for rewiring (see Section 4.2.1). Additionally, our planned IR or ultrasonic sensor to confirm block pickup (see Section 4.3.2) was deemed unnecessary as the gripper could reliably grasp the block.

Overall, the gripper mechanism worked very well for the attempted runs. Performance could have been further improved with integration of an autonomous block detection and approach routine.

3.4 Integration

Our primary issue when integrating all subsystems was instability with our power distribution, which often masked other problems (such as unreliable hardware or imprecise drivetrain calibration) and greatly limited our ability to develop and test the rover during later milestones. Since our power management setup went through so many iterations and the exact architecture of our components was so fragile, we did not trust that we could safely validate each system separately – to adequately test the gripper, we felt it had to be assembled, wired, and mounted, which prevented us from working on other tasks such as localization or motor tuning simultaneously.

For M3, many of the electrical architecture changes appeared to resolve these issues, though our robot was still subject to intermittent Bluetooth communication failures or brownouts upon pinging the LiDAR or ultrasonic sensors, requiring manual power cycles. In M3, we were able to successfully achieve a run integrating all subsystems of our design: we successfully avoided obstacles under manual control, localized and navigated to the delivery zone, and picked up and delivered the block to our designated dropoff zone, all with minimal manual intervention and no power cycling. However, because we were not able to completely diagnose our power distribution issues, our success in this run is due in significant part to good fortune with the behavior of our electronics rather than truly perfect integration. Thus, our overall performance here could have been further improved by fully resolving the underlying electrical issues, as well as improving our speed at completing all objectives or expanding the scope of our objectives, such as by automating various tasks.

4 Discussion

4.1 Best Features

4.1.1 Gripper

Our gripper was the most successful subsystem on the robot; having sound and well-scoped initial design with quick integration with the rest of the rover.

The wrist subsystem proved effective and reliable, providing a secure and stable grasp on the block. After adding padding to the contact interface, the block did not slip or drop during transport. The wrist can open to a wide angle, allowing tolerance for block positioning; however, drivetrain motor mounts limited how wide we could practically open the wrist. If we had been able to integrate and test the ultrasonic block sensing functionality, we are confident the clearance provided by the gripper would have allowed for successful autonomous pick-up of the block. Overall, the wrist subsystem was highly reliable and provided repeatable performance results across multiple test runs, reducing the need for corrective maneuvers during pick-up.

The arm subsystem generated sufficient torque to lift the gripper and raise the block, but some integration challenges occurred. The servo driving the arm occasionally slipped within its collar under load, causing inconsistencies in positioning and requiring manual realignment. Additionally, DC motor encoders and wiring near the back of the gripper prevented full arm retraction. As a result, the lift motion was reconfigured to rotate forward and upward, which increased the rover's footprint and required careful manual control. Despite these limitations, the gripper assembly remained operational and effective, with the wrist subsystem particularly being a reliable design element.

4.1.2 Drivetrain and Chassis

Our fundamental chassis design proved highly effective across all milestones. Navigating tight maze corners and corridors required minimizing the footprint to facilitate navigation and localization within a reasonable margin of error, especially during manual control. With the smallest corridors being 12" wide, we optimized the robot to under 8", providing up to 2" clearance on all sides in the tightest sections. The fundamental objective of minimizing the robot's footprint drove other mechanical design decisions, including chassis geometry, motor selection, and the gripper's actuation and structure. The Kiwi drivetrain was selected for the benefits of holonomic movement and for requiring fewer motors than traditional four-wheel or H-drive drivetrains, allowing for more compact designs. Layers of the chassis plates were fastened using 40mm bolts and nuts, allowing rapid disassembly and reassembly without specialized tools. Additionally, the layered design allowed unobstructed 360° LiDAR vision, and easy access to both Arduino USB ports for fast debugging.

Although the drivetrain proved reliable, several improvements could be implemented. Mounting electronics and routing wires presented significant challenges due to the tight footprint. Disconnecting or reconnecting wires often required substantial disassembly; replacing faulty motors or encoders necessitated disassembly of multiple systems, prohibiting testing

and debugging of other independent systems. More careful planning for the placement of electronics and the use of multiple wire channels could improve clearance and organization of cables. Currently, our current design features one main central channel for wire routing across all layers, which results in a large bundled mass in the center that makes individual wire identification difficult. Instead, we could implement multiple smaller channels for different wire routes to better isolate component-specific wiring.

When integrating electronic components, some improvements were identified with the design. Some wires extended beyond the chassis perimeter, causing snags against walls and connection faults. For example, the UART communication cables between the Uno and Mega were left exposed for ease of debugging and development, but any loosening of these wires during the milestones via accidental collisions with walls would result in catastrophic breakdowns in our system's functionality. Protective body paneling could have been implemented to prevent wire protrusion, however frequent debugging and disassembly made it impractical to add another assembly step.

4.1.3 Modular Programming Architecture

Our modular firmware architecture proved to be very effective and efficient when integrating additional functionality. By M2, we had solidified the architecture of a `Taskmaster` on each microcontroller managing a primary communications interface and a set of `Controller`s sending and receiving different messages, with those Controllers managing sensor or actuator interfaces (like a "drivetrain" or "gripper"), and those interfaces controlling the low-level sensor drivers (like the L298N or servo motors). The implementation of this architecture was somewhat time-consuming but required very little modification after M2.

By carefully stratifying this functionality, it allowed for very simple debugging when any incorrect behaviour was observed on the rover. For example, when a `Controller` was unresponsive to commands, we could isolate the issue to the execution of control logic rather than any problems with the communications pipeline, as the pipeline was the same for each `Controller`. As well, the abstraction of the `Translate` module (which converted structs and enums into `Message` objects) allowed for extremely simple and efficient creation of new message types, allowing for quick firmware prototypes of the LiDAR, ultrasonic, and gripper controllers. This structure also allowed for flexibility with which microcontrollers housed which subsystems in firmware, allowing us to seamlessly move the drivetrain controls between the Mega and Uno across milestones.

This ambitious approach did prove overly complex or onerous at times. Each `Controller` required significant memory overhead, allocating large chunks of heap memory for each runtime `MessageQueue` and demanding significant flash memory for code to manage the robust controls logic, communications interfaces, and low-level drivers. As a result, we had to consistently move control logic (such as for the drivetrain) between microcontrollers and tune the size of our queues and Messages to stay within our memory limits. Fortunately, all tunable parameters were easily accessible in a single settings file, and the abstraction of each `Controller` made moving their implementation extremely simple, allowing for rapid debugging and resolution of these issues. Although potentially overkill for a project of this scale, our sound programming design philosophy allowed us to quickly prototype and isolate various

rover failures as mechanical and electrical issues.

4.2 Worst Features

4.2.1 Power Management

As discussed previously (Sections 2.1.1, 2.2.1, 2.3.1, 2.4.1), our robot's performance across several milestones suffered due to flaws with our power management which we were unable to completely debug and resolve over the course of the project. In particular, we suspected the high voltage draw LiDAR as our main culprit and we recognized the critical importance of having constant voltage for the Bluetooth module.

Our strategy to verify functionality and efficiently debug individual components was to isolate each component to its own power source (Appendix A.6). In total, we used a 12V battery with three LM2596 voltage converters, a 9V battery with one L7805 linear voltage regulator (and 2 0.1 μF capacitors), and the 5V out pin of the Uno to provide reliable power to each component. Ultimately, this electrical set up was capable of providing steady and separate voltage sources to the Bluetooth, LiDAR, and other components.

Additionally, we worried about the jumper wires being accidentally pulled out of the breadboard or the female ports of the arduino boards during testing. To minimize this possibility, we used hot glue directly to the breadboard, masking tape and zip ties to securely fasten our connections and clean up wiring for aesthetics. Future improvements could be made with a PCB, screw terminals or JST connectors.

4.3 Alternative Designs

4.3.1 H-Drive

We initially considered implementing an H-drive configuration due to its holonomic capabilities, which would have provided superior maneuverability in the confined maze environment. This design would have utilized two drive motors, one mounted on each side of the chassis, along with a central strafe motor positioned perpendicular to the drive wheels. One significant advantage of this configuration was that the open central section would have provided substantial clearance and mounting space for the gripper mechanism.

However, we ultimately decided against this approach for several reasons. First, the H-drive configuration would have resulted in a considerably larger overall footprint, which conflicted with our primary design objective of minimizing robot dimensions. Second, properly mounting the strafe motor to ensure adequate traction and prevent wheel slippage presented significant mechanical challenges, particularly given our space constraints and the need for consistent ground contact pressure. Additionally, the H-drive configuration would have resulted in less optimal weight distribution, with mass concentrated along the perimeter rather than centralized.

Our final three-wheel Kiwi drive configuration provided the same holonomic movement while achieving the smallest possible footprint for our components. The kiwi drive's triangular wheel arrangement not only minimized the overall chassis dimensions but also provided

superior weight distribution with the mass centralized around the robot's center of gravity. This centralized weight distribution improved stability and control compared to the H-drive's peripheral weight placement, which would have positioned the drive motors and associated components toward the outer edges of the chassis.

During our design process, we drew inspiration from various VEX robotics competition designs, which often demonstrate elegant solutions to mechanical challenges within strict size and weight constraints. The modular construction approaches and efficient space utilization strategies commonly seen in VEX robots influenced several aspects of our chassis design and component mounting solutions.

4.3.2 Autonomous State Management

Due to time constraints, we were unable to implement an autonomous state machine on the rover as initially intended. In our proposal, we intended to have the rover automatically move between localization, block detection, and block delivery tasks, communicating automatically with the laptop to update its localization update and get drivetrain commands to move to its destination while avoiding obstacles (Appendix C.3.1). In addition to the firmware features we designed but were unable to integrate (i.e. the ultrasonics and automation control interface), this design would have integrated an IMU, magnetometer, and infrared sensor to redundantly monitor the robot's state and reset its progress in the state machine if a system failure was identified.

We planned to consistently monitor the IMU's accelerometer and compare this with the expected readings given our current control input. For example, if we command the rover to drive forward, we expect a constant acceleration along one axis, but if we see a spike in acceleration, it may indicate we have bumped into a wall – similarly, the magnetometer's compass would cross-reference our expected rotation commands to ensure our drivetrain responded appropriately. Once in the loading zone, an infrared sensor pointed at the gripper could also confirm if we successfully grasped the block, prompting the robot to try again if it failed. We intended to allow these redundant sensors to force the rover to reinitiate localization if they detected consistent anomalies.

This is an industry-standard firmware engineering design pattern, often called "model-based plausibility checks", and this philosophy is vital for any kind of autonomous system. Unfortunately, it was outside of the scope of this project given the course schedule and the time constraints to complete the maze. Since our maze runs were limited to five minutes, it proved far more useful to simply tune the behaviour of each subsystem to be as precise as possible under manual control, rather than actively monitor all of these fault and state reset cases as we would for a safety critical autonomous system such as a self-driving car.

References

[1] Components101.com. Hc-05 bluetooth module datasheet. URL: https://components101.com/sites/default/files/component_datasheet/HC-05%20Datasheet.pdf.

[2] Components101.com. Mg90s – metal gear micro servo motor. URL: https://components101.com/motors/mg90s-metal-gear-servo-motor.

[3] Texas Instruments. Lm2596 simple switcher® power converter 150-khz 3-a step-down voltage regulator datasheet, 2005. URL: https://www.ti.com/lit/ds/symlink/lm2596.pdf.

[4] ST Microelectronics. L298n dual full-bridge driver, 2023. URL: https://www.st.com/resource/en/datasheet/l298.pdf.

[5] Robopeak. rplidar_arduino. URL: https://github.com/robopeak/rplidar_arduino.git.

[6] SLAMTEC. Rplidar a-series, a1m8 datasheet. URL: https://www.slamtec.com/en/Support#rplidar-a-series.

[7] Arduino S.r.l. Arduino® mega 2560 rev3 datasheet. SKU A000067, November 2025. URL: https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf.

[8] Arduino S.r.l. Arduino® uno r3 / arduino® uno boards datasheet. SKU A000066, November 2025. URL: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf.

[9] T. Teng, T. H. Ha, Z. Yang, and S. Colic. Mie444 lab 5_2025. Department of Mechanical and Industrial Engineering, University of Toronto, September 2020.

A Electrical

A.1 Initial Proposal Design

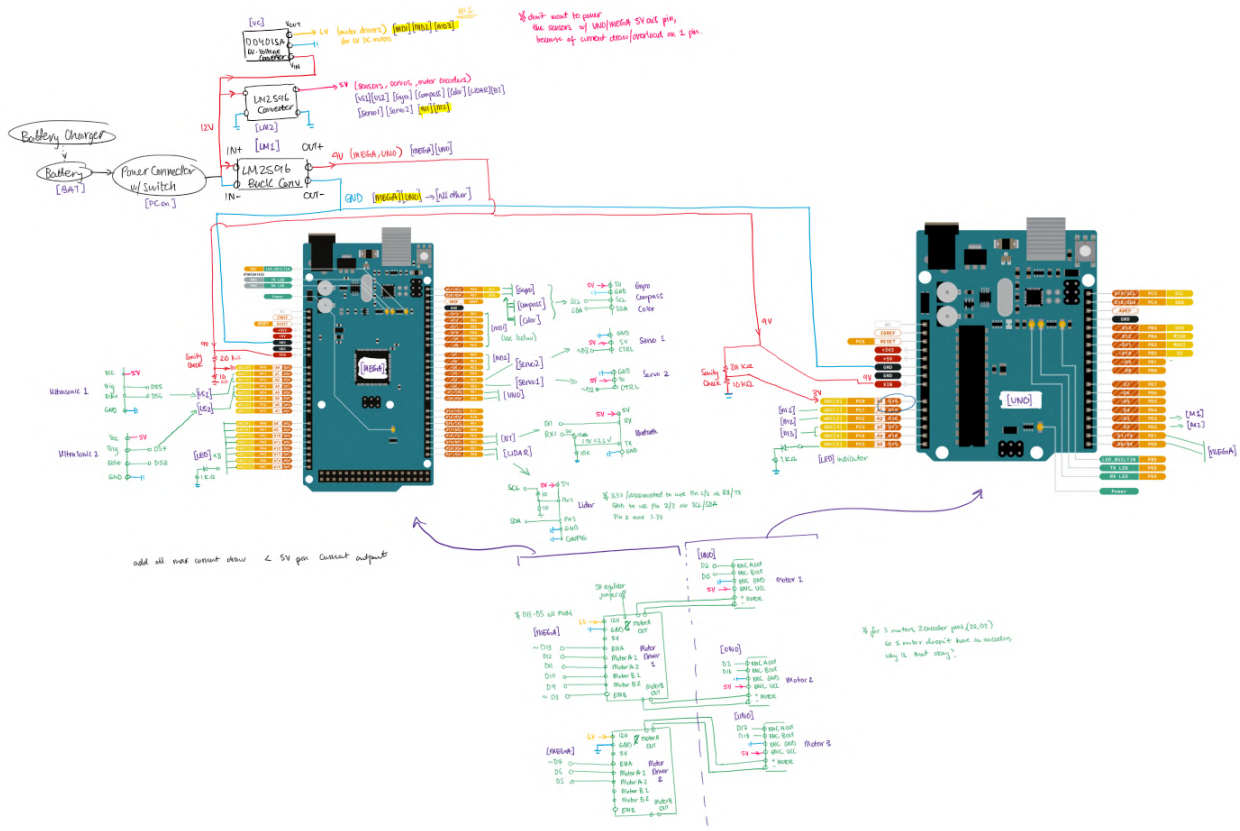


Figure 1: Motors, Encoders, Bleutooth, LiDAR, Servos, Sensors (Ultrasonic, Colour, Compass, Gyroscope) on Arduino Mega and Uno

A.2 Milestone 1

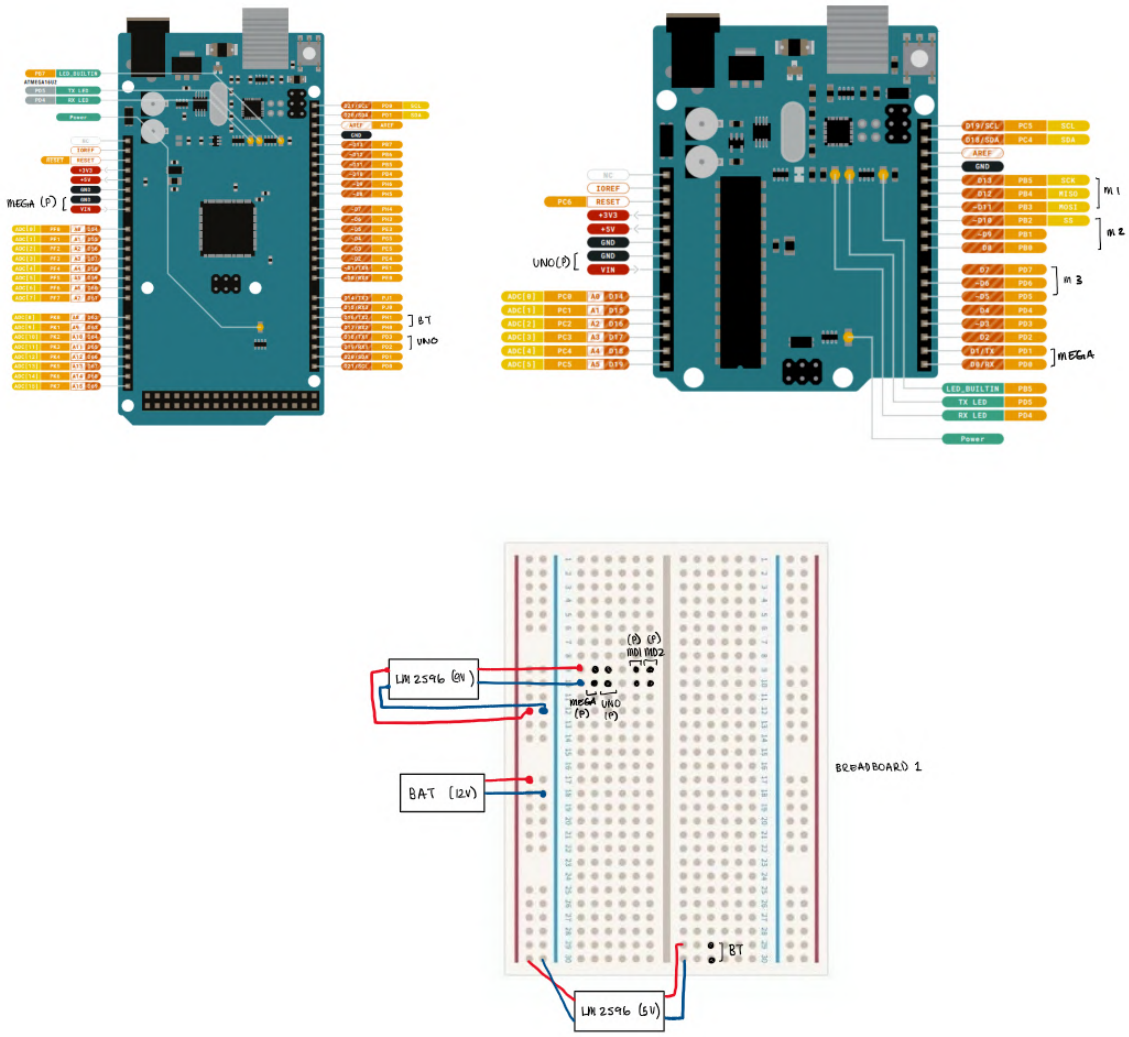


Figure 2: Motors, Bluetooth on Arduino Mega and Uno

A.3 Milestone 2

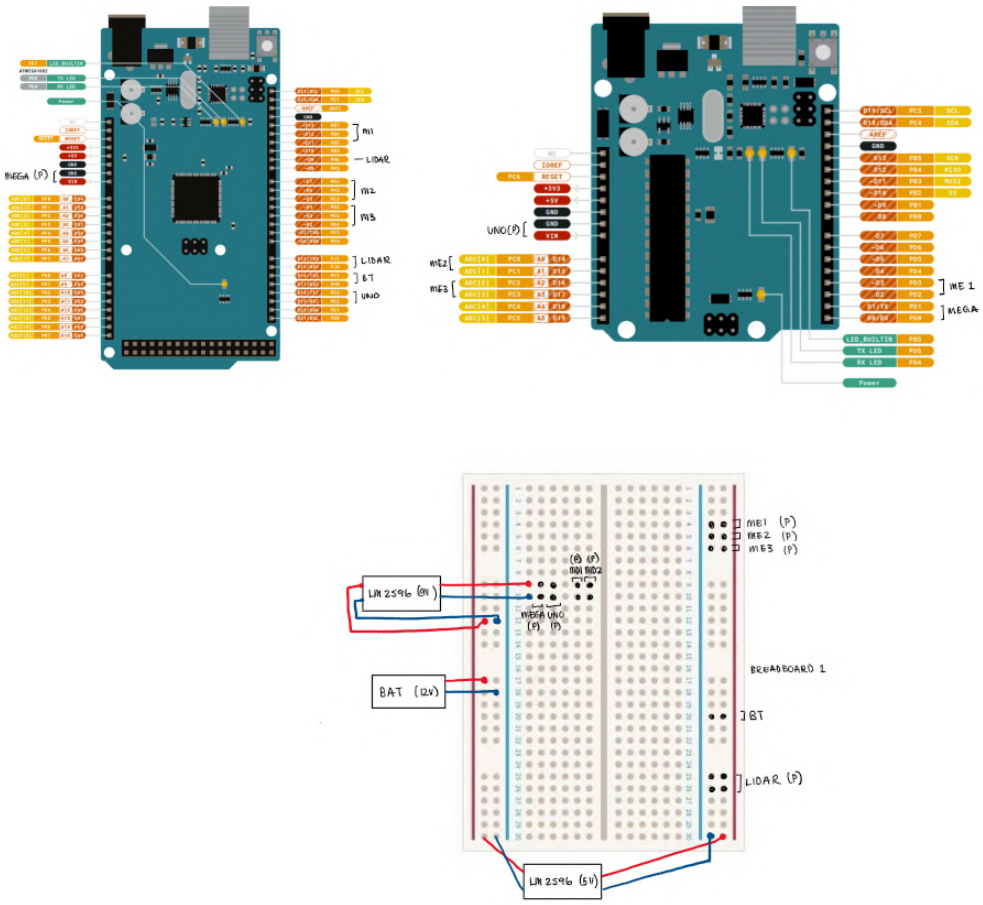


Figure 3: Motors, Encoders, Bluetooth, LiDAR on Arduino Mega and Uno

A.4 Milestone 3

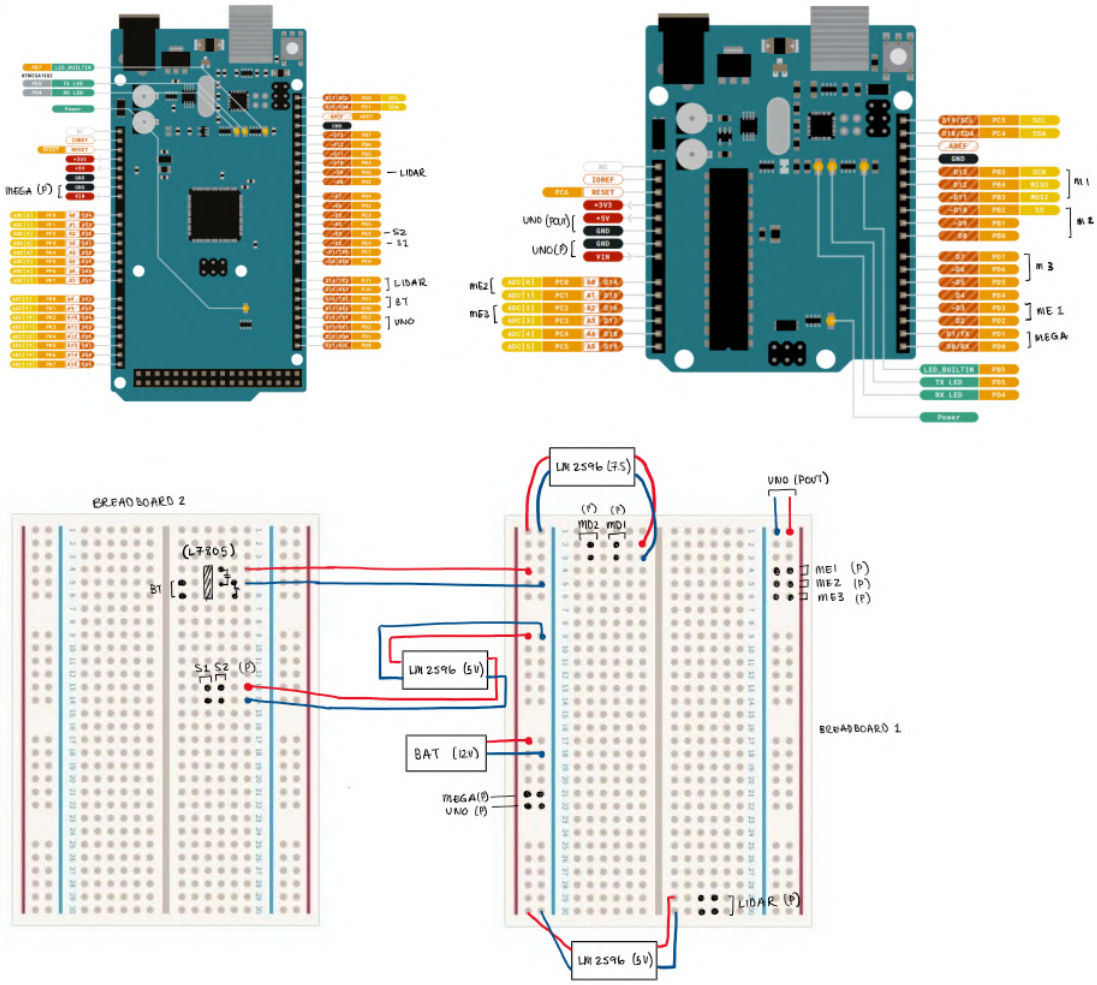


Figure 4: Motors, Encoders, Bluetooth, LiDAR, Servos on Arduino Mega and Uno

A.5 Individual Wiring Diagram

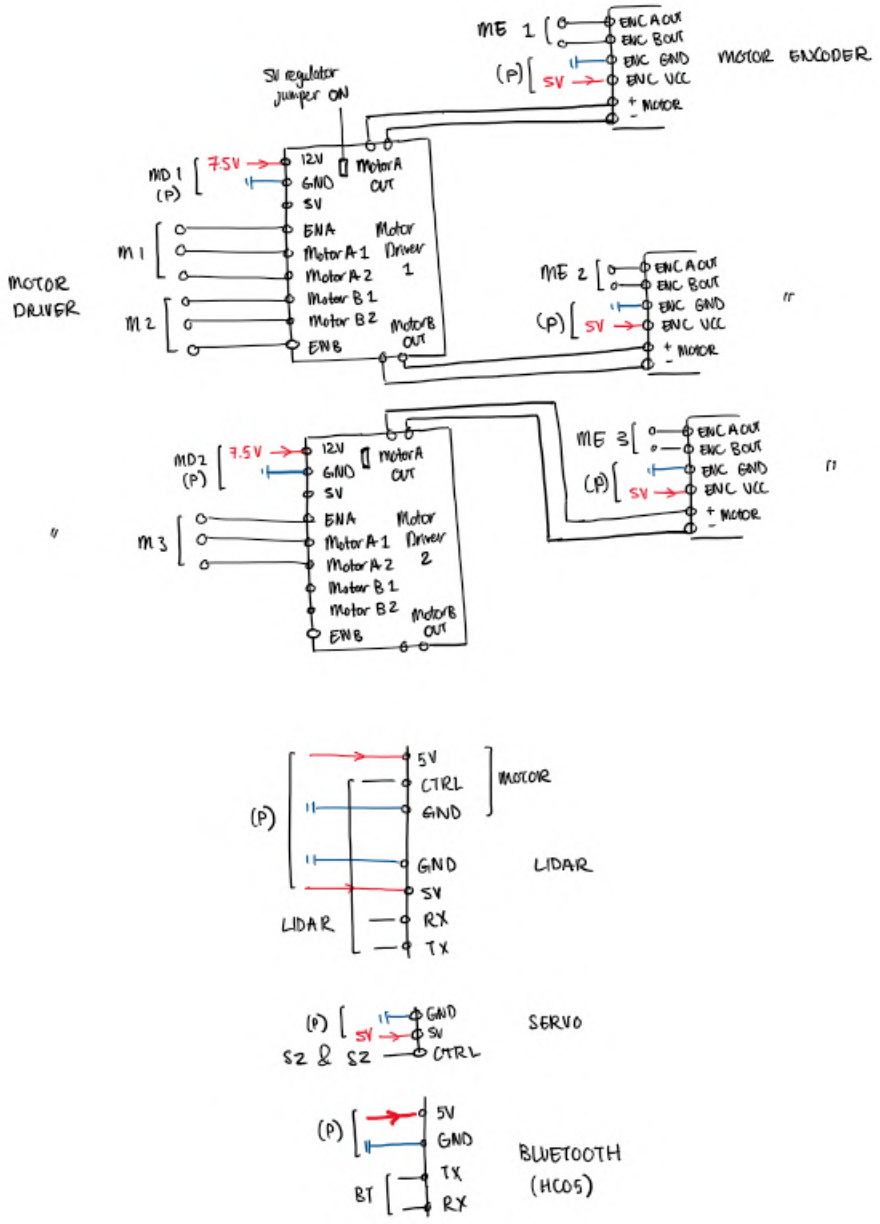


Figure 5: Motors, Encoders, Motor Drivers, LiDAR, Bluetooth, Servos

A.6 Power Distribution

Final Power Distribution	
Power Source	Component (being powered) [Appendix A3 label]
12V Battery [BAT1]	Mega [MEGA (P)] Uno [UNO (P)] [LM2596-1, 7.5V] [LM2596-2, 5V]
[LM2596-1, 7.5V]	Motor Drivers [MD1, MD2]
[LM2596-2, 5V]	LiDAR [LIDAR]
[LM2596-3, 5V]	2 Servos [S1, S2]
9V Battery [BAT2]	[L7805, 5V]
[L7805, 5V]	Bluetooth [BT]
UNO, 5V Vout [UNO POUT]	3 Motor Encoders [ME1 (P),ME2 (P), ME3 (P)]

Figure 6: Power Distribution setup for Milestone 3

B Mechanical

B.1 Drivetrain and Chassis

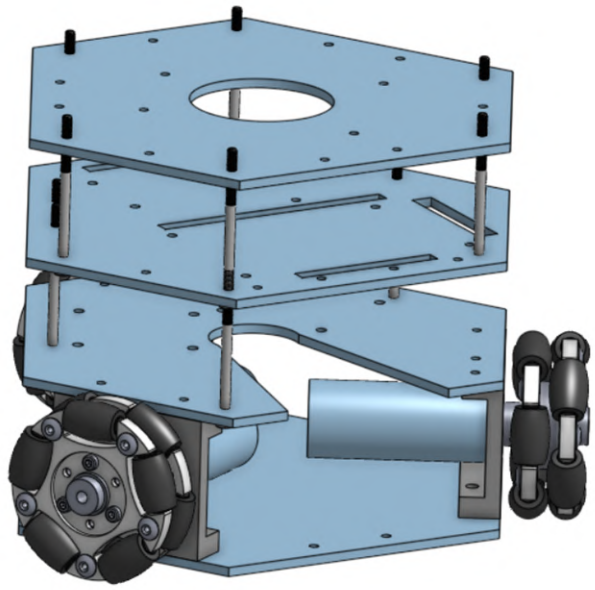


Figure 7: Drivetrain and Chassis CAD, side view

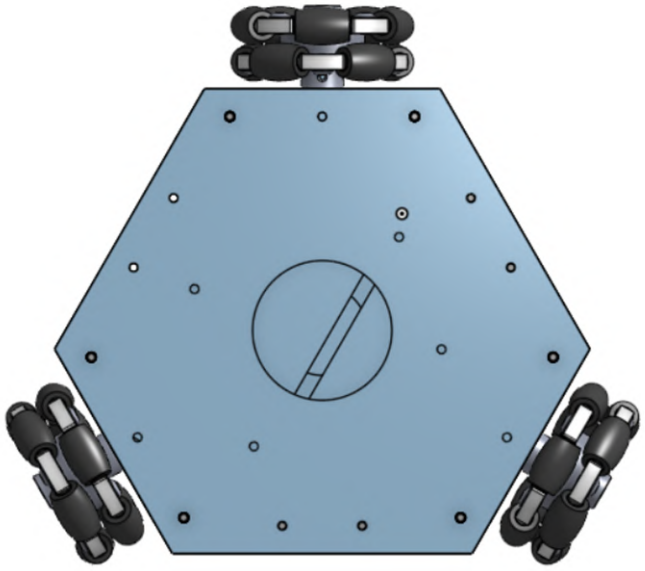


Figure 8: Drivetrain and Chassis CAD, bird's eye view

B.2 Gripper Assembly, CAD

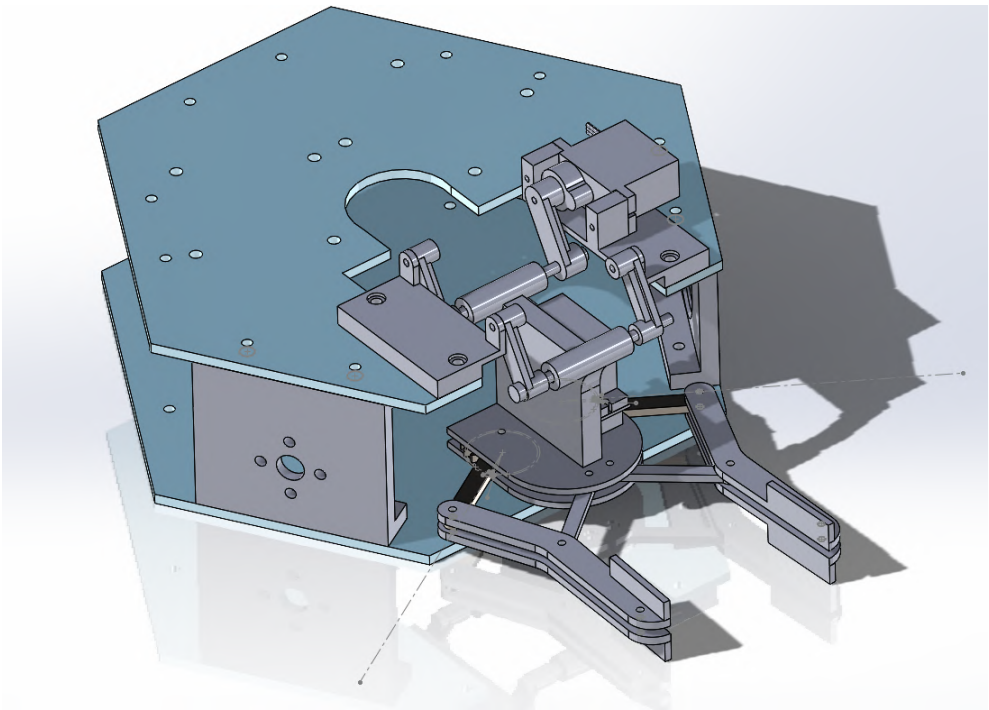


Figure 9: Full Gripper Assembly

B.3 Wrist Subsystem, CAD

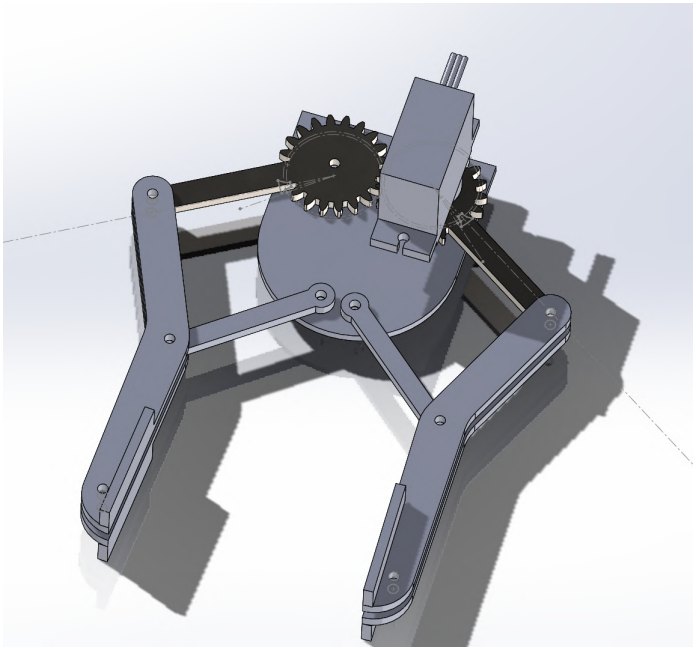


Figure 10: Gripper Wrist subsystem CAD

B.4 Arm Subsystem, CAD

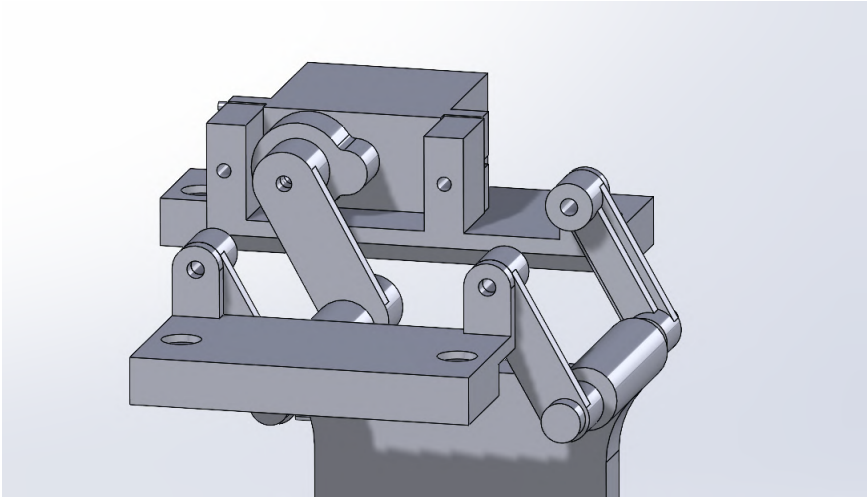


Figure 11: Gripper Arm subsystem CAD

B.5 Gripper Assembly, Final

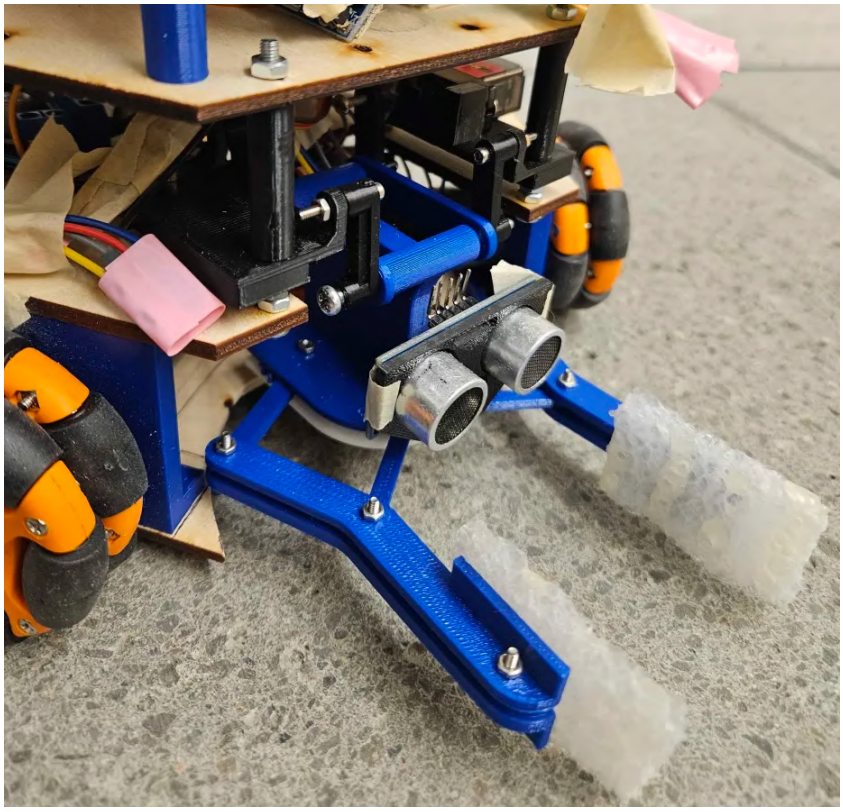


Figure 12: Gripper Assembly with Hex Nut spacers and Plywood Cutout, side view

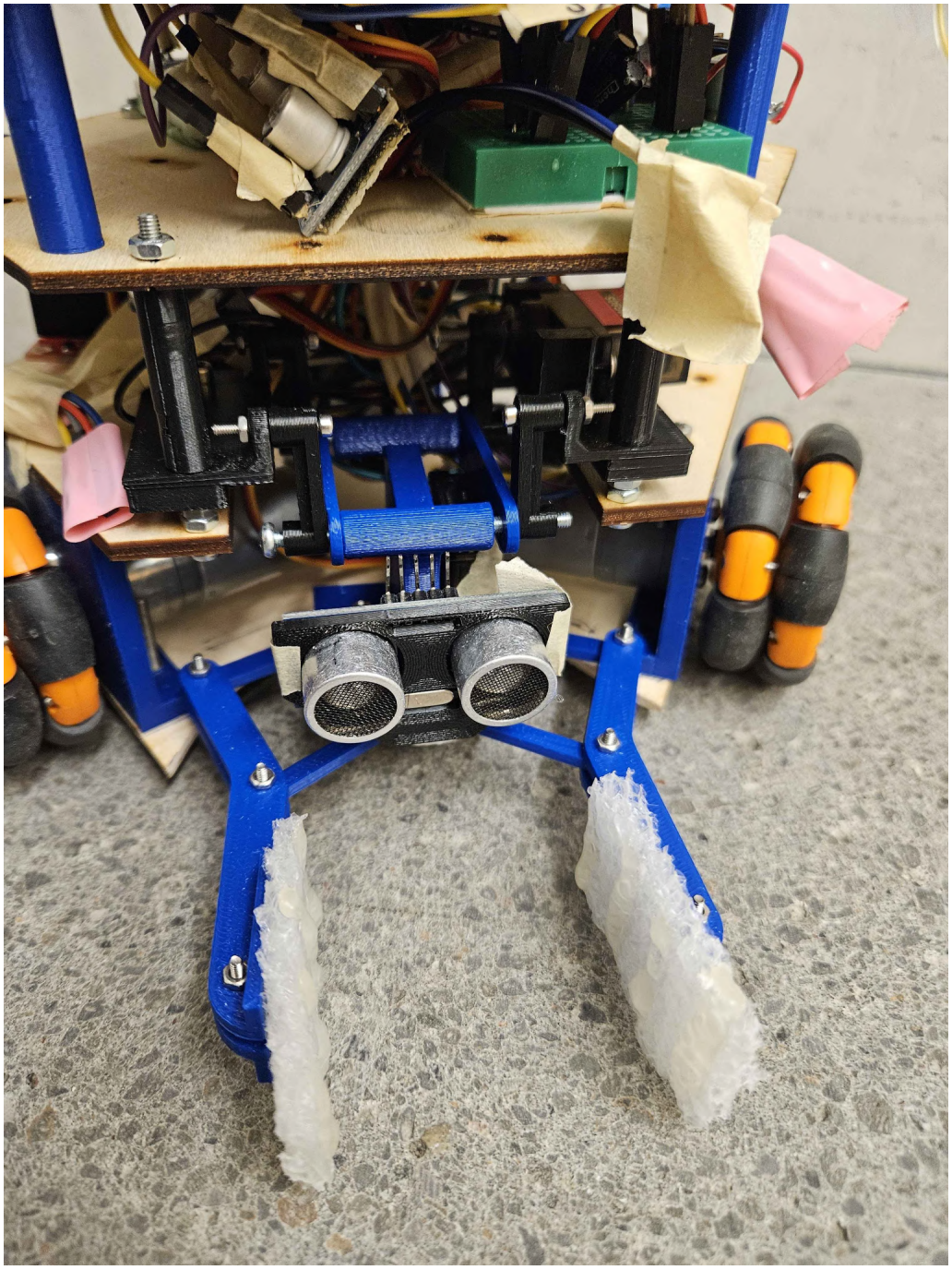


Figure 13: Gripper Assembly with Hex Nut spacers and Plywood Cutout, front view

B.6 Full Assembly, CAD

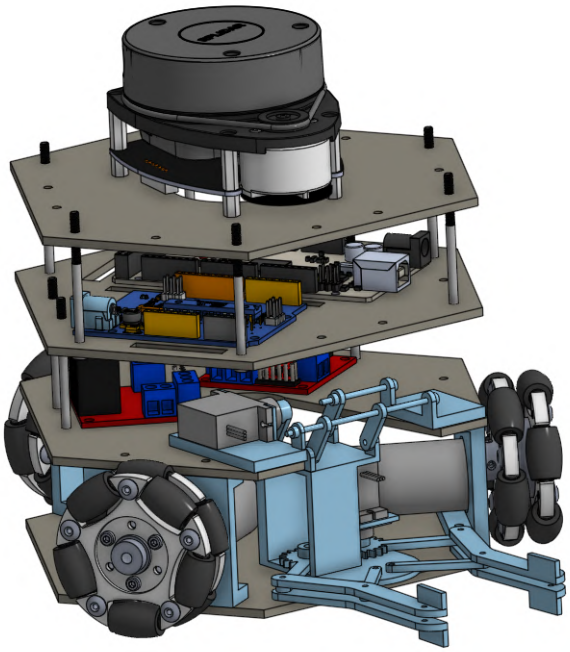


Figure 14: Full Assembly CAD, front view

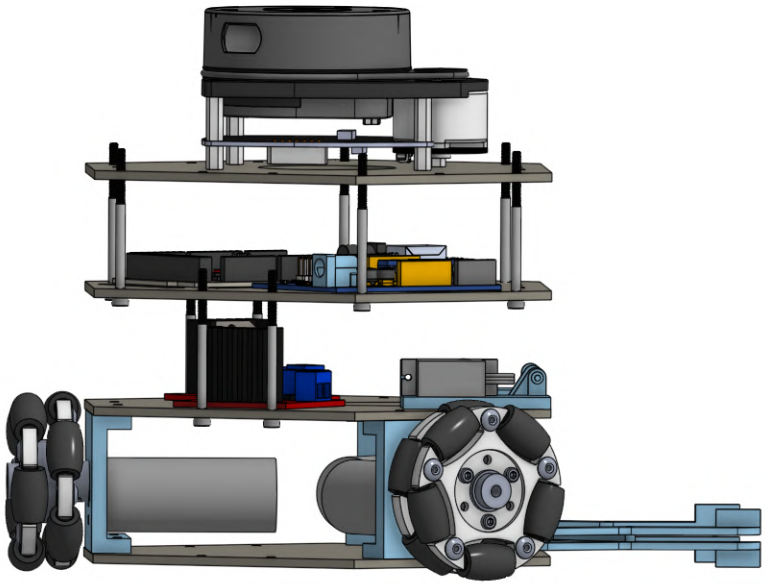


Figure 15: Full Assembly CAD, side view

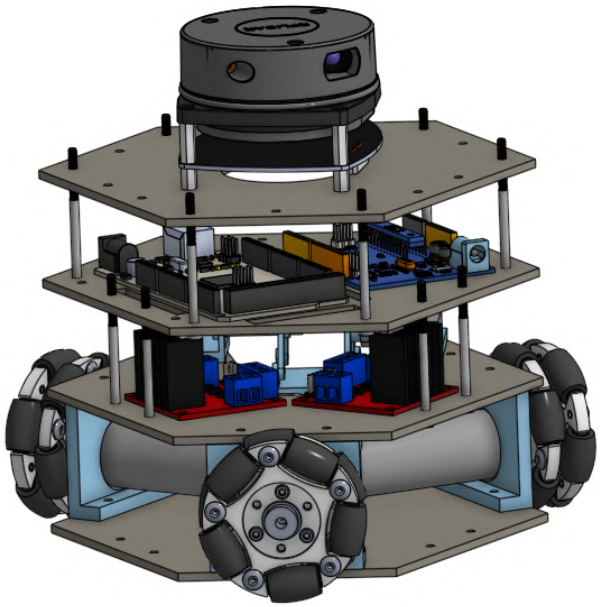


Figure 16: Full Assembly CAD, rear view

C Code Excerpts

C.1 Communications Interface

C.1.1 Translate Interface, Milestone 1

```
File = include/DrivetrainCommands.h
commit = 4bcf24fe28b0f34d1e054f6fbb2d1a58ae65e3ae

/**
 * String literal messages for drivetrain communications
 */
#define DRIVETRAIN_COMMAND_TRANSLATE_FORWARD "w$\0"
#define DRIVETRAIN_COMMAND_TRANSLATE_BACKWARD "s$\0"
#define DRIVETRAIN_COMMAND_ROTATE_LEFT "a$\0"
#define DRIVETRAIN_COMMAND_ROTATE_RIGHT "d$\0"
#define DRIVETRAIN_COMMAND_HALT "q$\0"

/**
 * @brief Convert drivetrain command to string.
 *
 * @param command
 * @param buffer
 */
static inline void drivetrainCommandAsString(DrivetrainCommands command, char* buffer)
{
   switch (command)
   {
      case DrivetrainCommands::TranslateForward:
         memoryCopy(buffer, DRIVETRAIN_COMMAND_TRANSLATE_FORWARD, MESSAGE_LENGTH_MAX);
         break;
      case DrivetrainCommands::TranslateBackward:
         memoryCopy(buffer, DRIVETRAIN_COMMAND_TRANSLATE_BACKWARD, MESSAGE_LENGTH_MAX);
         break;
      case DrivetrainCommands::RotateLeft:
         memoryCopy(buffer, DRIVETRAIN_COMMAND_ROTATE_LEFT, MESSAGE_LENGTH_MAX);
         break;
      case DrivetrainCommands::RotateRight:
         memoryCopy(buffer, DRIVETRAIN_COMMAND_ROTATE_RIGHT, MESSAGE_LENGTH_MAX);
         break;
      case DrivetrainCommands::Halt:
         memoryCopy(buffer, DRIVETRAIN_COMMAND_HALT, MESSAGE_LENGTH_MAX);
         break;
      default:
         memorySet(buffer, 0, MESSAGE_LENGTH_MAX);
   }
}

/**
 * @brief Interpret string as drivetrain command.
 *
 * @param buffer
 * @return DrivetrainCommands
 */
static inline DrivetrainCommands stringAsDrivetrainCommand(char* buffer)
{
   if (stringLength(buffer) == 0)
      return DrivetrainCommands::NoReceivedCommand;

   if (stringCompare(buffer, DRIVETRAIN_COMMAND_TRANSLATE_FORWARD) == true)
      return DrivetrainCommands::TranslateForward;
   if (stringCompare(buffer, DRIVETRAIN_COMMAND_TRANSLATE_BACKWARD) == true)
      return DrivetrainCommands::TranslateBackward;
   if (stringCompare(buffer, DRIVETRAIN_COMMAND_ROTATE_LEFT) == true)
      return DrivetrainCommands::RotateLeft;
   if (stringCompare(buffer, DRIVETRAIN_COMMAND_ROTATE_RIGHT) == true)
```

```
        return DrivetrainCommands::RotateRight;
    if (stringCompare(buffer, DRIVETRAIN_COMMAND_HALT) == true)
        return DrivetrainCommands::Halt;

    return DrivetrainCommands::InvalidCommand;
}
```

Code Listing 1: Message to Enum Lookup Table

C.1.2 Translate Interface, Final

```
File = lib/Translate/Translate.h

/**
 * @brief Instantiate all EnumMessageMap objects for translation
 *
 * Each call to ENUM_MESSAGE_MAP_TRANSLATION(enumType)
 * creates enumTypeTranslation
 */
ENUM_MESSAGE_MAP_TRANSLATION(DrivetrainManualCommand)
ENUM_MESSAGE_MAP_TRANSLATION(DrivetrainManualResponse)
ENUM_MESSAGE_MAP_TRANSLATION(DrivetrainAutomatedResponse)
ENUM_MESSAGE_MAP_TRANSLATION(DrivetrainEncoderState)

/**
 * @brief Instantiate all StructMessageMap objects for translation
 *
 * Each call to STRUCT_MESSAGE_MAP_TRANSLATION(structType)
 * creates structTypeTranslation.
 *
 * All structs MUST be marked as __attribute__((packed))
 */
STRUCT_MESSAGE_MAP_TRANSLATION(DrivetrainEncoderDistances)
STRUCT_MESSAGE_MAP_TRANSLATION(DrivetrainAutomatedCommand)
STRUCT_MESSAGE_MAP_TRANSLATION(DrivetrainDisplacements)
STRUCT_MESSAGE_MAP_TRANSLATION(DrivetrainMotorCommand)

File = lib/Translate/TranslateEnumDefs.h

template <typename E, size_t numElements>
class EnumMessageMap {
private:
    MessageType type;
    const EnumStringMap (&enumStringMap)[numElements];

    /**
     * @brief Given a generic enum, get the string literal representation.
     *
     * @param e An enum
     * @return constexpr const char* String literal representation
     */
    const char* enumToStr(E e) const {
        for (size_t i = 0; i < numElements; ++i)
            if (enumStringMap[i].enumValue == static_cast<uint8_t>(e))
                return enumStringMap[i].strRep;
        return "";
    }

    /**
     * @brief Given a string, get the generic enum representation.
     *
     * @param buffer A string
     * @return constexpr E Generic enum representation
     */
    const E strToEnum(const char* buffer) const {
        if (!buffer || buffer[0] == '\0')
```

```
            return E::NoReceived;

        for (size_t i = 0; i < numElements; ++i)
            if (stringsEqual(enumStringMap[i].strRep, buffer))
                return static_cast<E>(enumStringMap[i].enumValue);

        return E::Invalid;
    }

public:
    EnumMessageMap(MessageType type, const EnumStringMap (&map)[numElements])
        : type(type), enumStringMap(map) {};

    /**
     * @brief Provided an enum, initialize and output a corresponding message.
     *
     * @param e An enum to translate
     * @param outMessage Message with appropriate type and content, now initialized
     */
    void asMessage(const E e, Message* outMessage) {
        outMessage->init(type, MESSAGE_CONTENT_SIZE_AUTOMATIC, enumToStr(e));
    }

    /**
     * @brief Provided a message, return a corresponding enum.
     *
     * @param message A message to translate
     * @return E An appropriate enum
     */
    E asEnum(Message* message) {
        char buffer[MESSAGE_CONTENT_LENGTH_MAX];
        message->getContent(buffer);
        return strToEnum(buffer);
    }
};

File = lib/Translate/TranslateStructDefs.h

/*
 * Define a StructMessageMap to connect a struct to a given MessageType.
 */

template <typename S>
class StructMessageMap {
private:
    MessageType type;
    size_t size;

public:
    StructMessageMap(MessageType type)
        : type(type), size(sizeof(S)) {};

    /**
     * @brief Given a generic value, get the char representation (i.e. as bytes).
     *
     * @param s A pointer to a struct of type S
     * @param out A buffer of chars, of known final size based on sizeof(S)
     */
    void structToStr(const S* s, char* out) const {
        memoryCopy(out, s, this->size);
    }

    /**
     * @brief Given a string (i.e. bytes), get the struct representation.
     *
     * @param s A pointer to a struct of type S
     * @param buffer A string, of known initial size based on sizeof(S)
     *
```

```
     * Must be instantiated by each individual class. See Translate.h
     */
    void strToStruct(S* s, const char* buffer) const = delete;

    /**
     * @brief Provided a pointer to a struct, initialize and output a corresponding message.
     *
     * @param s A pointer to a struct to translate
     * @param outMessage Message with appropriate type and content, now initialized
     */
    void asMessage(const S* s, Message* outMessage) {
        char buffer[MESSAGE_CONTENT_LENGTH_MAX];
        structToStr(s, buffer);
        outMessage->init(type, this->size, buffer);
    }

    /**
     * @brief Provided a message and pointer to a struct
     *
     * @param message A message to translate
     * @param s Pointer to a struct
     */
    void asStruct(Message* message, S* s) {
        char buffer[MESSAGE_CONTENT_LENGTH_MAX];
        message->getContent(buffer);
        strToStruct(s, buffer);
    }
};

File = lib/Translate/Translate.cpp

template<>
void StructMessageMap<DrivetrainEncoderDistances>::strToStruct(
    DrivetrainEncoderDistances* s,
    const char* buffer) const
{
    const uint8_t* asBytes = (const uint8_t*)buffer;

    s->encoder1Dist = (encoderDistance_in)deserializeF32(asBytes);
    s->encoder2Dist = (encoderDistance_in)deserializeF32(asBytes);
    s->encoder3Dist = (encoderDistance_in)deserializeF32(asBytes);
}
```

Code Listing 2: Generalized Enum and Struct to Enum Serialization/Deserialization

C.1.3 Translate Interface, Python

```
File = python/controller/message.py

# Registry for the format of all received content.
#
# Similar to Translate.h
_TYPE_FORMATS = {
    MessageType.DrivetrainEncoderDistances: dict(
        fmt="<fff",  # three float32_t
        units=("in", "in", "in"),  # three inches
        disp=["{:.2f}␣{u}", "{:.2f}␣{u}", "{:.2f}␣{u}"],  # display format
    ),
    MessageType.DrivetrainDisplacements: dict(
        fmt="<fff",  # three float32_t
        units=("in", "in", "  "),  # delta x, delta y, delta theta
        disp=[
            "{:.2f}␣{u}",
            "{:.2f}␣{u}",
            lambda v, u: f"{v␣*␣RAD_TO_DEG:.2f}␣{u}",  # rad deg for display
            ],  # display format
```

```
    ),
    MessageType.DrivetrainMotorCommand: dict(
        fmt="<BfBfBf",  # three bool then float32_t
        units=("", "", "", "", "", ""),  # unitless
        disp=[
            lambda v, u: f"{'+' if v == 1 else '-'}{u}",
            "{:.2f}{u}"
            ]*3,  # display format
    ),
    MessageType.DrivetrainAutomatedCommand: dict(
        fmt="<hhh",  # three int16_t
        units=("in", "in", "  "),  # delta x, delta y, delta theta
        disp=["{:.2f} {u}", "{:.2f} {u}", "{:.2f} {u}"],  # display format
    ),
    MessageType.LidarPointReading: dict(
        fmt="<hh",  # two int16_t
        units=("  ", "in"),  # degree, in
        disp=["{}{u}", "{} {u}"],  # display format
    ),
    MessageType.UltrasonicPointReading: dict(
        fmt="<Bffff", # a uint8_t, four float32_t
        units=("", "in", "in", "in", "in"), # which , three encoders, ultrasonic
        disp=["{}{u}", "{:.2f} {u}", "{:.2f} {u}", "{:.2f} {u}", "{:.2f} {u}"]
    ),
    MessageType.Generic: dict(text=True),
    MessageType.Error: dict(text=True),
}

# Message class
#
# Corresponds to lib/Message/Message.h
class Message:

    def __init__(self, type: MessageType, content: bytes):
        self.type = type
        self.content = content
        self.size = len(content)
        self.raw = self.encode()

    def get_type(self):
        return self.type

    def get_content(self):
        return self.content

    @classmethod
    def from_raw(self, raw: bytes):
        if len(raw) < ENCODING_MINIMUM_LENGTH:
            raise ValueError("Raw message too short")

        # Extract first two bytes
        type_val = raw[0]
        size = raw[1]

        # Decode type
        try:
            msg_type = MessageType(type_val)
        except ValueError:
            raise ValueError(f"Invalid message type value: {type_val}")

        # Extract content and validate terminator
        content = raw[2 : 2 + size]

        # Validate the end char
        expected_end = raw[2 + size : 3 + size]
        if expected_end != MESSAGE_END_CHAR:
            raise ValueError(f"Missing or invalid message end char: {expected_end}")
```

```python
        # Remove null terminator if present
        raw = raw.rstrip(NULL_TERMINATOR)

        # Construct message
        msg = self(msg_type, content)
        msg.raw = raw
        return msg

    def encode(self) -> bytes:
        return (
            self.type.value.to_bytes(1, "little")
            + self.size.to_bytes(1, "little")
            + self.content
            + MESSAGE_END_CHAR
        )

    def _decode_struct(self, meta):
        fmt = meta["fmt"]
        expected = struct.calcsize(fmt)
        if len(self.content) != expected:
            raise ValueError(
                f"{self.type.name}: expected {expected} bytes, got {len(self.content)}"
            )
        values = struct.unpack(fmt, self.content)
        return values

    def decode(self):
        meta = _TYPE_FORMATS.get(self.type)
        if not meta:
            # no known structure
            return self.content.decode(errors="replace") if self.content else None
        if meta.get("text"):
            return self.content.decode(errors="replace")
        if "fmt" in meta:
            return self._decode_struct(meta)
        return self.content

    def __repr__(self):
        meta = _TYPE_FORMATS.get(self.type)
        val = self.decode()
        if meta and "fmt" in meta:
            # metadata will include formatting method
            disp, units = meta["disp"], meta["units"]
            parts = []
            for i, v in enumerate(val):
                fmt = (
                    disp[i]
                    if isinstance(disp, (list, tuple)) and i < len(disp)
                    else disp
                )
                unit = units[i] if i < len(units) else ""
                if callable(fmt):
                    parts.append(fmt(v, unit))    # <-- NEW
                else:
                    parts.append(fmt.format(v, u=unit))

            return f"<{self.type.name}({', '.join(parts)})>"

        if meta and meta.get("text"):
            # display raw text representation
            return f"<{self.type.name}({val})>"

        if not self.content:
            # no content to display
            return f"<{self.type.name}()>"

        # raw bytes representation as last resort
        try:
```

```
            return f"<{self.type.name}({self.content.decode(errors='replace')})>"
        except Exception:
            return f"<{self.type.name}({self.content!r})>"
```

Code Listing 3: Corresponding Python deserialization logic

C.2 Control Logic

C.2.1 Drivetrain Logic, Milestone 1

```
File = lib/DriveController/DriveController.cpp
commit = 4bcf24fe28b0f34d1e054f6fbb2d1a58ae65e3ae

/**
 * @brief Apply command to drivetrain.
 *
 * @param command
 * @param drivetrain
 */
void DriveController::applyCommand(DrivetrainCommands command, Drivetrain* drivetrain)
{
   switch (command)
   {
      case DrivetrainCommands::TranslateForward:
         drivetrain->setTranslate(DRIVETRAIN_TRANSLATE_SPEED, true);
         break;
      case DrivetrainCommands::TranslateBackward:
         drivetrain->setTranslate(DRIVETRAIN_TRANSLATE_SPEED, false);
         break;
      case DrivetrainCommands::RotateLeft:
         drivetrain->setRotate(DRIVETRAIN_ROTATE_SPEED, true);
         break;
      case DrivetrainCommands::RotateRight:
         drivetrain->setRotate(DRIVETRAIN_ROTATE_SPEED, false);
         break;
      case DrivetrainCommands::Halt:
      default:
         drivetrain->halt();
         break;
   }
}

/**
 * @brief Receive commands from the comms interface and issue to Drivetrain. Send back an
 * appropriate acknowledge.
 *
 * @param commsInterface
 * @param drivetrain
 */
void DriveController::processCommsForDrivetrain(CommsInterface* comms, Drivetrain* drivetrain)
{
   DrivetrainCommands currentCommand = readCommsForCommand(comms);

   // Send acknowledgement if invalid command
   if (currentCommand == DrivetrainCommands::InvalidCommand)
   {
      this->sendResponse(comms, DrivetrainResponses::AcknowledgeInvalidCommand);
      return;
   }

   // Send echo and acknowledgement if valid command
   if (currentCommand != DrivetrainCommands::NoReceivedCommand)
   {
      arbitrateCommand(currentCommand, drivetrain);
      this->sendCommandEcho(comms, currentCommand);
```

```
      this->sendResponse(comms, DrivetrainResponses::AcknowledgeValidCommand);
      return;
   }

   // Check if should halt based on no recent commands
   if (this->shouldHalt())
   {
      arbitrateCommand(DrivetrainCommands::Halt, drivetrain);
      this->sendCommandEcho(comms, DrivetrainCommands::Halt);
      this->sendResponse(comms, DrivetrainResponses::NotifyHalting);
      return;
   }
}
```

Code Listing 4: Original drivetrain control logic

C.2.2 Forward and Inverse Kinematics

```
File = lib/Drivetrain/DrivetrainDefs.h

static const float32_t A[9] = {
   ( 0.000000), (-1.000000), ( 3.569000),
   (-0.866025), ( 0.500000), ( 3.569000), // A[2,2] negated for y-sign convention
   ( 0.866025), ( 0.500000), ( 3.569000), // A[3,2] negated for y-sign convention
};

static const float32_t Ainv[9] = {
    ( 0.000000), (-0.577350), ( 0.577350),
    (-0.666666), ( 0.333333), ( 0.333333),
    ( 0.093396), ( 0.093396), ( 0.093396)
};

/**
 * @brief Inverse kinematics. Get drivetrain displacement from encoder readings.
 *
 * @param displacements Updated after call
 * @param encodersBefore
 * @param encodersAfter
 */
static inline void displacementsFromEncoderReadings(
   DrivetrainDisplacements *displacements,
   DrivetrainEncoderDistances *encodersBefore,
   DrivetrainEncoderDistances *encodersAfter
)
{
   if (!displacements || !encodersBefore || !encodersAfter)
      return;

   // Compute encoder displacements
   encoderDistance_in d1 = encodersAfter->encoder1Dist - encodersBefore->encoder1Dist;
   encoderDistance_in d2 = encodersAfter->encoder2Dist - encodersBefore->encoder2Dist;
   encoderDistance_in d3 = encodersAfter->encoder3Dist - encodersBefore->encoder3Dist;

   // Compute displacements
   displacements->dX_in      = (Ainv[0] * d1) + (Ainv[1] * d2) + (Ainv[2] * d3);
   displacements->dY_in      = (Ainv[3] * d1) + (Ainv[4] * d2) + (Ainv[5] * d3);
   displacements->dTheta_rad = (Ainv[6] * d1) + (Ainv[7] * d2) + (Ainv[8] * d3);
}

/**
 * @brief Forward kinematics. Get updated encoder readings after displacement.
 *
 * @param displacements
 * @param encodersBefore
 * @param encodersAfter Updated after call
 */
```

```
static inline void encoderReadingsFromDisplacement(
   DrivetrainDisplacements *displacements,
   DrivetrainEncoderDistances *encodersBefore,
   DrivetrainEncoderDistances *encodersAfter
)
{
   if (!displacements || !encodersBefore || !encodersAfter)
      return;

   // Compute displacements
   const float32_t d1 = \
      (A[0] * displacements->dX_in) +
        (A[1] * displacements->dY_in) +
        (A[2] * displacements->dTheta_rad);
   const float32_t d2 = \
      (A[3] * displacements->dX_in) +
        (A[4] * displacements->dY_in) +
        (A[5] * displacements->dTheta_rad);
   const float32_t d3 = \
      (A[6] * displacements->dX_in) +
        (A[7] * displacements->dY_in) +
        (A[8] * displacements->dTheta_rad);

   // Compute encoder readings
   encodersAfter->encoder1Dist = encodersBefore->encoder1Dist + d1;
   encodersAfter->encoder2Dist = encodersBefore->encoder2Dist + d2;
   encodersAfter->encoder3Dist = encodersBefore->encoder3Dist + d3;
}
```

Code Listing 5: Drivetrain kinematics logic

C.2.3 PID Control Implementation

```
File = include/Settings.h

# define DRIVETRAIN_BACKWARDS_ADDED_GAIN_MOTOR_1 (1.3)
# define DRIVETRAIN_BACKWARDS_ADDED_GAIN_MOTOR_2 (1.7)
# define DRIVETRAIN_BACKWARDS_ADDED_GAIN_MOTOR_3 (1.9)

# define DRIVETRAIN_AUTOMATED_GAIN_KP_MOTOR_1 (75)
# define DRIVETRAIN_AUTOMATED_GAIN_KI_MOTOR_1 (23)
# define DRIVETRAIN_AUTOMATED_GAIN_KD_MOTOR_1 (200)

# define DRIVETRAIN_AUTOMATED_GAIN_KP_MOTOR_2 (33)
# define DRIVETRAIN_AUTOMATED_GAIN_KI_MOTOR_2 (10)
# define DRIVETRAIN_AUTOMATED_GAIN_KD_MOTOR_2 (170)

# define DRIVETRAIN_AUTOMATED_GAIN_KP_MOTOR_3 (68)
# define DRIVETRAIN_AUTOMATED_GAIN_KI_MOTOR_3 (17)
# define DRIVETRAIN_AUTOMATED_GAIN_KD_MOTOR_3 (145)

# define DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS (3.75)
# define DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS (6)
# define DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS (0.05)
# define DRIVETRAIN_ALLOWED_OVERSHOOT (0.7)

File = lib/Drivetrain/DrivetrainDefs.h

/**
 * @brief Compute the drivetrain motor command from the encoder values now and at target.
 *
 * @param encodersNow
 * @param encodersTarget
 * @param command // updated after call
 */
static inline void getDrivetrainMotorCommand(
```

```
    DrivetrainEncoderDistances *encodersNow,
    DrivetrainEncoderDistances *encodersTarget,
    DrivetrainMotorCommand *command,
     bool isFirstCommand // Resets integral control
)
{
    // Compute inverse kinematics
    DrivetrainDisplacements displacements;
    displacementsFromEncoderReadings(
       &displacements,
       encodersNow,
       encodersTarget
    );

    // Compute encoder delta
    DrivetrainEncoderDistances dEncoders;
     dEncoders.encoder1Dist = (encodersTarget->encoder1Dist - encodersNow->encoder1Dist);
     dEncoders.encoder2Dist = (encodersTarget->encoder2Dist - encodersNow->encoder2Dist);
     dEncoders.encoder3Dist = (encodersTarget->encoder3Dist - encodersNow->encoder3Dist);

     // Update control variables
     static time_ms lastMillis = millis();
     static DrivetrainEncoderDistances dEncodersInt = {0};
     static DrivetrainEncoderDistances dEncodersLast = {0};
     static DrivetrainEncoderDistances dEncodersDiff = {0};
    static DrivetrainMotorCommand prevCommand = {
         .is1Forward=false, .speed1=0,
         .is2Forward=false, .speed2=0,
         .is3Forward=false, .speed3=0
     };
     if (isFirstCommand)
     {
         lastMillis = millis();
         dEncodersInt = {0};
         dEncodersLast = dEncoders;
         prevCommand = {0};
     }
     else
     {
         // Compute time step
         time_ms now = millis();
         time_s dt = MS_TO_S(now - lastMillis);

         if (dt > DRIVETRAIN_MINIMUM_DT_PID)
         {
             /// Update integral and derivative terms if enough time elapsed
             dEncodersInt.encoder1Dist = constrain(
                 dEncodersInt.encoder1Dist + (dEncoders.encoder1Dist * dt),
                 -DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS,
                 DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS
             );
             dEncodersInt.encoder2Dist = constrain(
                 dEncodersInt.encoder2Dist + (dEncoders.encoder2Dist * dt),
                 -DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS,
                 DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS
             );
             dEncodersInt.encoder3Dist = constrain(
                 dEncodersInt.encoder3Dist + (dEncoders.encoder3Dist * dt),
                 -DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS,
                 DRIVETRAIN_MAXIMUM_INTEGRAL_MOTORS
             );

             /// Derivative
             dEncodersDiff.encoder1Dist = constrain(
                 (dEncoders.encoder1Dist - dEncodersLast.encoder1Dist) / dt,
                 -DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS,
                 DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS
             );
```

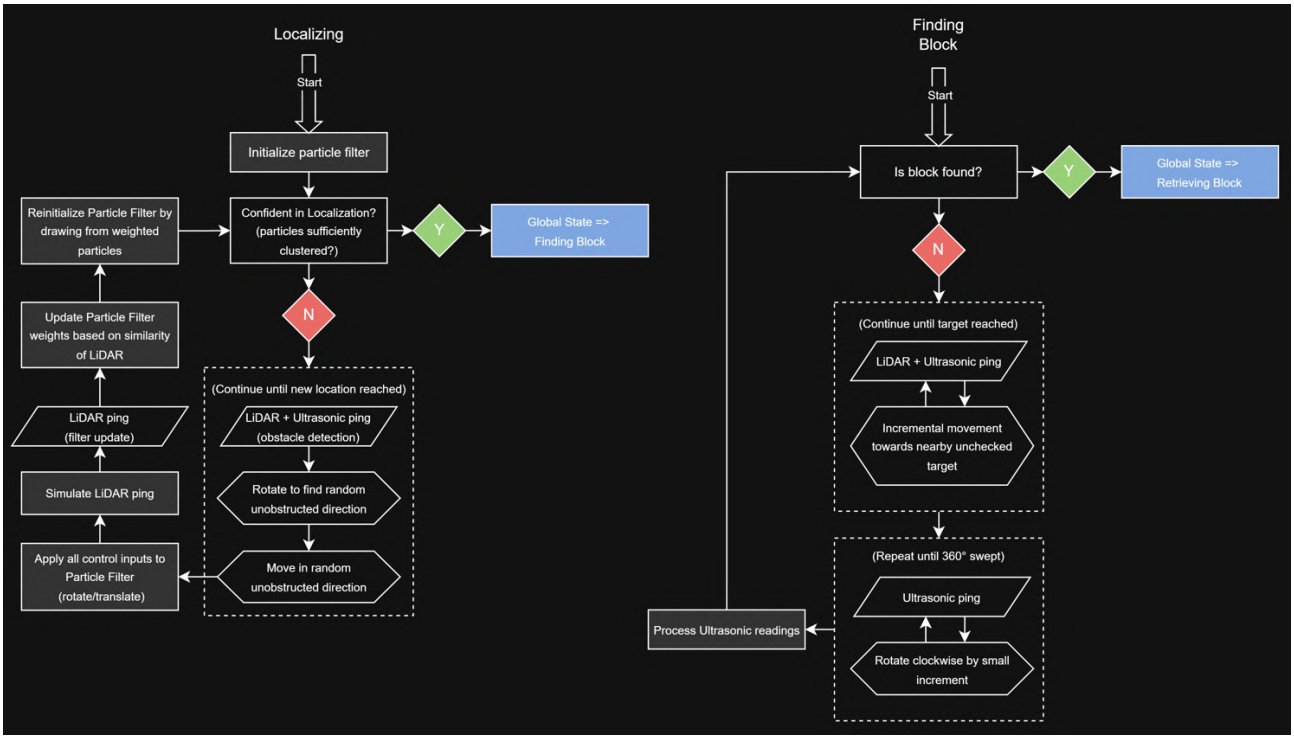
```
            dEncodersDiff.encoder2Dist = constrain(
                (dEncoders.encoder2Dist - dEncodersLast.encoder2Dist) / dt,
                -DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS,
                DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS
            );
            dEncodersDiff.encoder3Dist = constrain(
                (dEncoders.encoder3Dist - dEncodersLast.encoder3Dist) / dt,
                -DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS,
                DRIVETRAIN_MAXIMUM_DERIVATIVE_MOTORS
            );

            // Update static
            dEncodersLast = dEncoders;
            lastMillis = now;
        }
    }

    /// Limit impact of proportional
    dEncoders.encoder1Dist = constrain(
        dEncoders.encoder1Dist,
        -DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS,
        DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS
    );
    dEncoders.encoder2Dist = constrain(
        dEncoders.encoder2Dist,
        -DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS,
        DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS
    );
    dEncoders.encoder3Dist = constrain(
        dEncoders.encoder3Dist,
        -DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS,
        DRIVETRAIN_MAXIMUM_PROPORTIONAL_MOTORS
    );

    // Apply PID gains
    DrivetrainEncoderDistances controlDistances;
    controlDistances.encoder1Dist = \
        (dEncoders.encoder1Dist * DRIVETRAIN_AUTOMATED_GAIN_KP_MOTOR_1) +
        (dEncodersInt.encoder1Dist * DRIVETRAIN_AUTOMATED_GAIN_KI_MOTOR_1) +
        (dEncodersDiff.encoder1Dist * DRIVETRAIN_AUTOMATED_GAIN_KD_MOTOR_1);

    controlDistances.encoder2Dist = \
        (dEncoders.encoder2Dist * DRIVETRAIN_AUTOMATED_GAIN_KP_MOTOR_2) +
        (dEncodersInt.encoder2Dist * DRIVETRAIN_AUTOMATED_GAIN_KI_MOTOR_2) +
        (dEncodersDiff.encoder2Dist * DRIVETRAIN_AUTOMATED_GAIN_KD_MOTOR_2);

    controlDistances.encoder3Dist = \
        (dEncoders.encoder3Dist * DRIVETRAIN_AUTOMATED_GAIN_KP_MOTOR_3) +
        (dEncodersInt.encoder3Dist * DRIVETRAIN_AUTOMATED_GAIN_KI_MOTOR_3) +
        (dEncodersDiff.encoder3Dist * DRIVETRAIN_AUTOMATED_GAIN_KD_MOTOR_3);

    // Compute special gains
    if (displacements.dX_in < 0) // Backwards gain
    {
        controlDistances.encoder1Dist *= DRIVETRAIN_BACKWARDS_ADDED_GAIN_MOTOR_1;
        controlDistances.encoder2Dist *= DRIVETRAIN_BACKWARDS_ADDED_GAIN_MOTOR_2;
        controlDistances.encoder3Dist *= DRIVETRAIN_BACKWARDS_ADDED_GAIN_MOTOR_3;
    }

    // Compute forward kinematics after gain
    motorCommandFromDistances(
       &controlDistances,
       command,
         (motorSpeedRaw)DRIVETRAIN_AUTOMATED_MIN_SPEED,
       (motorSpeedRaw)DRIVETRAIN_AUTOMATED_MAX_SPEED
    );

    // Apply slew
```

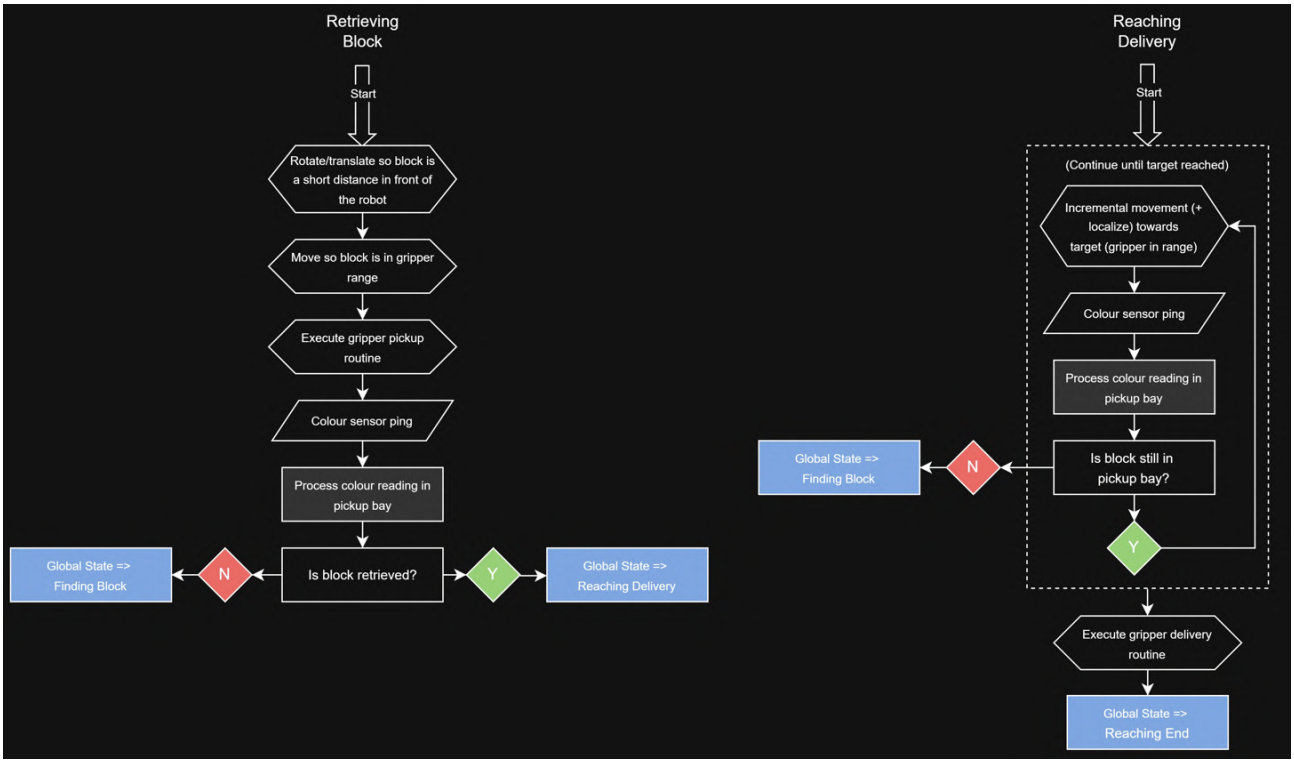
```
    command->speed1 = applySlewLimit(command->speed1, prevCommand.speed1);
    command->speed2 = applySlewLimit(command->speed2, prevCommand.speed2);
    command->speed3 = applySlewLimit(command->speed3, prevCommand.speed3);

    // Store previous command
     prevCommand = *command;
}
#endif
```

Code Listing 6: PID Control logic

C.3 Architecture

C.3.1 Initial State Diagram





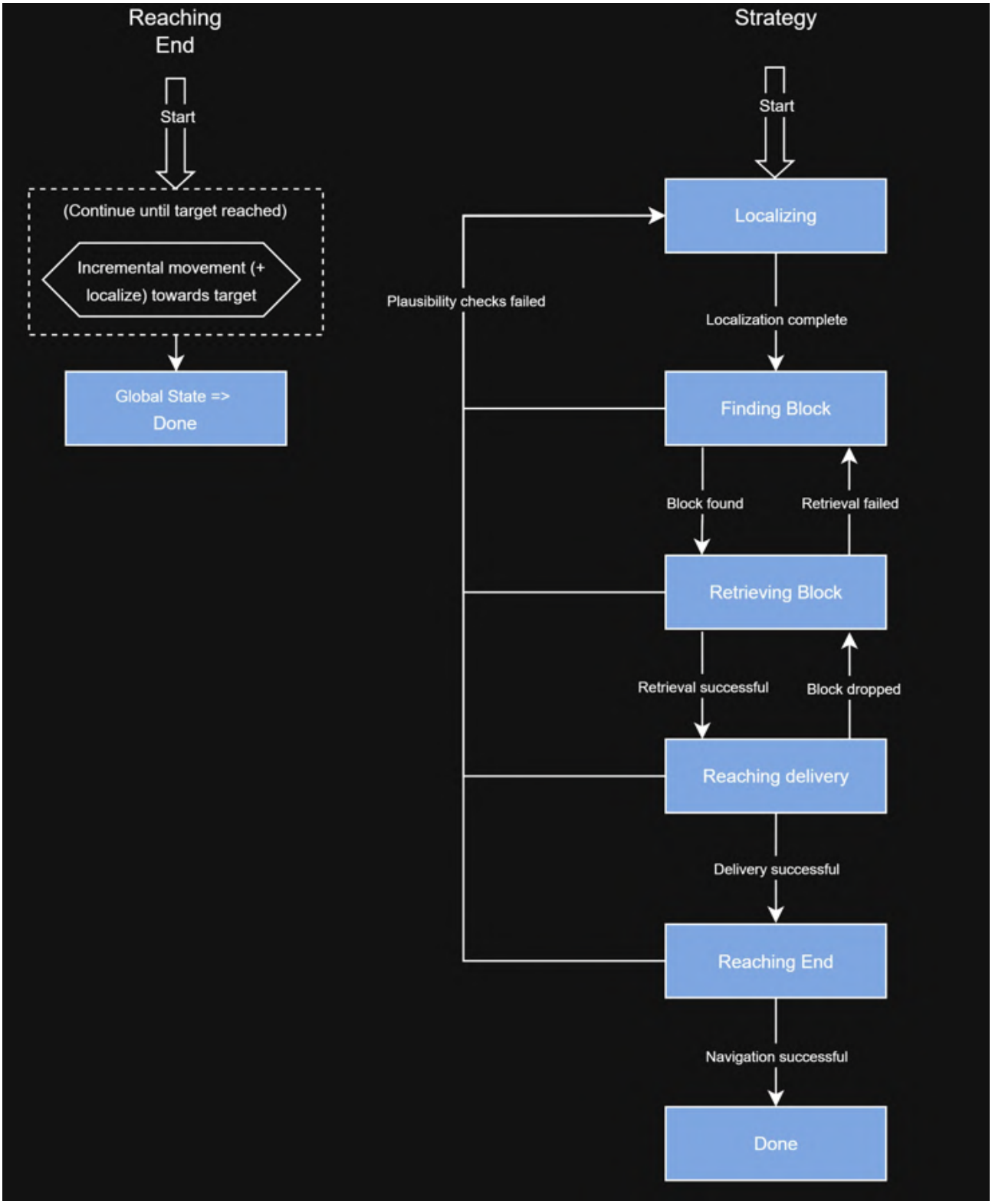


Figure 17: State diagrams for the final proposal

C.3.2 Hierarchical Structure

```
File = src/peripheral/Main.cpp

/*****************************************************
 *                    COMMUNICATIONS                 *
 *****************************************************/
static CommsInterface g_controllerComms;

/*****************************************************
 *                     CONTROLLERS                   *
 *****************************************************/
static DrivetrainEncoders g_drivetrainEncoders;
static DriveEncoderController g_driveEncoderController(&g_drivetrainEncoders);
Drivetrain g_drivetrain;
DriveController g_driveController(&g_drivetrain, &g_driveEncoderController);

/*****************************************************
 *                     TASKMASTERS                   *
 *****************************************************/
static ControllerGeneric* controllers[] = {
   &g_driveController,
   &g_driveEncoderController
};
TASKMASTER_DECLARE(taskmaster, &g_controllerComms, controllers)

/**
 * @brief Global setup functions for board
 */
void setup()
{
   Wiring_InitPins();
   Wiring_InitComms(&g_controllerComms);
   Wiring_InitDrivetrainEncoders(&g_drivetrainEncoders);
   Wiring_InitDrivetrain(&g_drivetrain);
}

/**
 * @brief Runtime loop for board
 */
void loop()
{
   taskmaster.execute();
}
```

Code Listing 7: Controller hierarchical structure on Arduino Uno

C.3.3 Taskmaster Operational Loop

```
File = lib/Taskmaster/Taskmaster.cpp

/**
 * @brief Execute a full loop of the Taskmaster.
 *
 */
void Taskmaster::execute(void)
{
   receive();

    if (this->hasExternalMessage)
    {
        dispatch(&this->externalMessage);
        this->hasExternalMessage = false;
    }

   Message message;
   while(poll(&message))
```

```
    {
        dispatch(&message);
    }

    process();
    collect();
}
```

Code Listing 8: Operational logic for Taskmaster

C.4 Localization

C.4.1 Particle Filter Weight Update

```
File = python/controller/mcl2/mcl_helper.py

def update_weight(
    self,
    simulated_reading: LidarReading,
    real_reading: LidarReading,
    min_weight: float = MIN_WEIGHT
):
    global certainty

    # Downsample simulated and real LIDAR readings for speed
    simulated_lidar_downsampled = simulated_reading.get_downsampled()
    real_lidar_downsampled = real_reading.get_downsampled()

    # Normalize readings to account for differences in modeled space
    simulated_lidar_normalized = simulated_lidar_downsampled.get_normalized()
    real_lidar_normalized = real_lidar_downsampled.get_normalized()

    # Extract downsample Lidar Point Readings
    simulated_lidar_points = simulated_lidar_normalized.get_points()
    real_lidar_points = real_lidar_normalized.get_points()

    # Construct dictionaries of points to identify angles contained in both sets
    # The real LIDAR reading may not always compute all angles
    simulated_lidar_map = {
        int(round(p.angle)) % 360: p.distance for p in simulated_lidar_points
    }
    real_lidar_map = {
        int(round(p.angle)) % 360: p.distance for p in real_lidar_points
    }
    angles_to_compare = sorted(
        set(simulated_lidar_map.keys()).intersection(set(real_lidar_map.keys()))
    )

    # If no angles are shared, weight particle as little as possible
    if len(angles_to_compare) == 0:
        self.weight = float(min_weight)
        return

    # Use log-likelihood to avoid underflows
    # Sum the log of the probability distribution functions
    log_likelihood = 0.0

    # Precompute gaussian coefficient term for speed: log(1/(sigma*sqrt(2pi)))
    sensor_std = MIN_SENSOR_STD_PERCENT + (MAX_SENSOR_STD_PERCENT - MIN_SENSOR_STD_PERCENT) *
        max(1 - certainty, 0)
    gaussian_log_prefactor = -0.5 * math.log(2.0 * math.pi * (sensor_std ** 2))

    # Iterate through angles and get log-likelihoods
    for ang in angles_to_compare:
        z_simulated = simulated_lidar_map[ang]   # predicted distance (inches)
```

```
        z_real = real_lidar_map[ang]     # observed distance (inches)

        # Compute log PDF of normal distribution
        raw_diff = z_simulated - z_real
        raw_sign = 1 if (z_simulated > z_real) else -1
        diff = (abs(raw_diff)**1.5) * raw_sign
        log_pdf = gaussian_log_prefactor - 0.5 * ((diff ** 2) / (sensor_std ** 2))
        log_likelihood += log_pdf

    # Convert back into weight space and clamp the minimum weight
    try:
        weight = math.exp(log_likelihood)
    except OverflowError:
        # If log_likelihood is too small/large, clamp to min_weight
        weight = float(min_weight)

    # Again, enforce floor to avoid degenerate zero
    self.weight = max(weight, float(min_weight))
```

Code Listing 9: Localization weight update function

C.4.2 Inverse Kinematics

```
File = python/controller/inverse_kinematics.py

# Robot parameters
L = 3.569  # inches - DISTANCE_FROM_OBJECT_CENTER_TO_WHEEL_MIDPOINT

DEG_TO_RAD = math.pi / 180.0
THETA_1 = 180 * DEG_TO_RAD  # radians
THETA_2 = 60 * DEG_TO_RAD  # radians
THETA_3 = 300 * DEG_TO_RAD  # radians

A = np.array([
    [-math.sin(THETA_1), math.cos(THETA_1), L],
    [-math.sin(THETA_2), math.cos(THETA_2), L],
    [-math.sin(THETA_3), math.cos(THETA_3), L]
])

# Inverse of A
A_INV = np.linalg.inv(A)

def inverse_kinematics(initial: EncoderReading, current: EncoderReading):
    """
    Compute delta_x, delta_y, delta_theta between two encoder readings.
    """
    inital_readings = initial.get_readings()
    final_readings = current.get_readings()
    motor_displacements = np.array([
        inital_readings[0] - final_readings[0],
        inital_readings[1] - final_readings[1],
        inital_readings[2] - final_readings[2],
    ])
    Robot_displacement = A_INV @ motor_displacements
    delta_x, delta_y, delta_theta =
        Robot_displacement[0], Robot_displacement[1], Robot_displacement[2]
    return delta_x, delta_y, delta_theta
```

Code Listing 10: Localization inverse kinematics mathematics

C.4.3 LiDAR Visualization

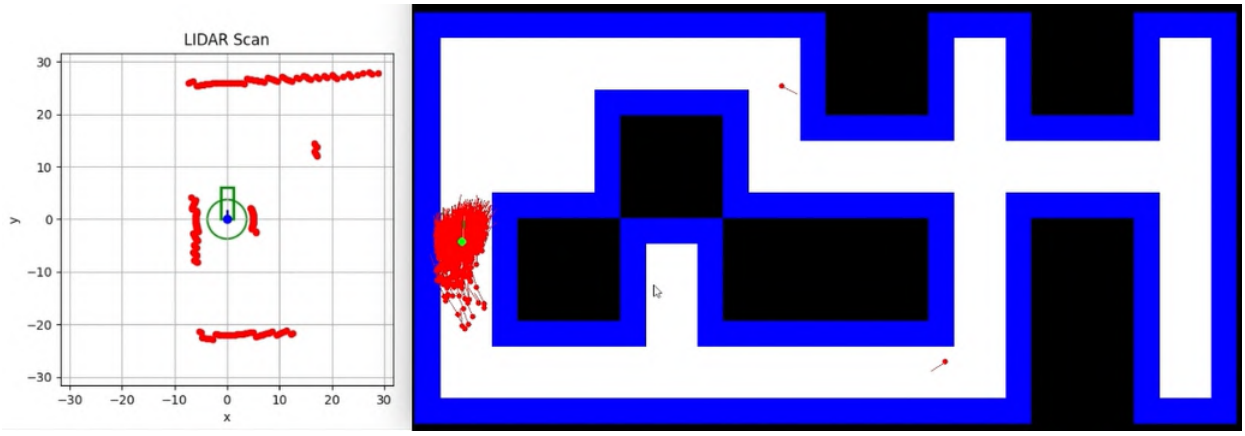


Figure 18: LiDAR Visualization with Particle Filter. Red particles in particle filter indicate possible state, green particle is estimated state. Red dots in LiDAR visualization are observed data points.

D Kiwi Drive Mathematics

D.1 Movement Diagrams

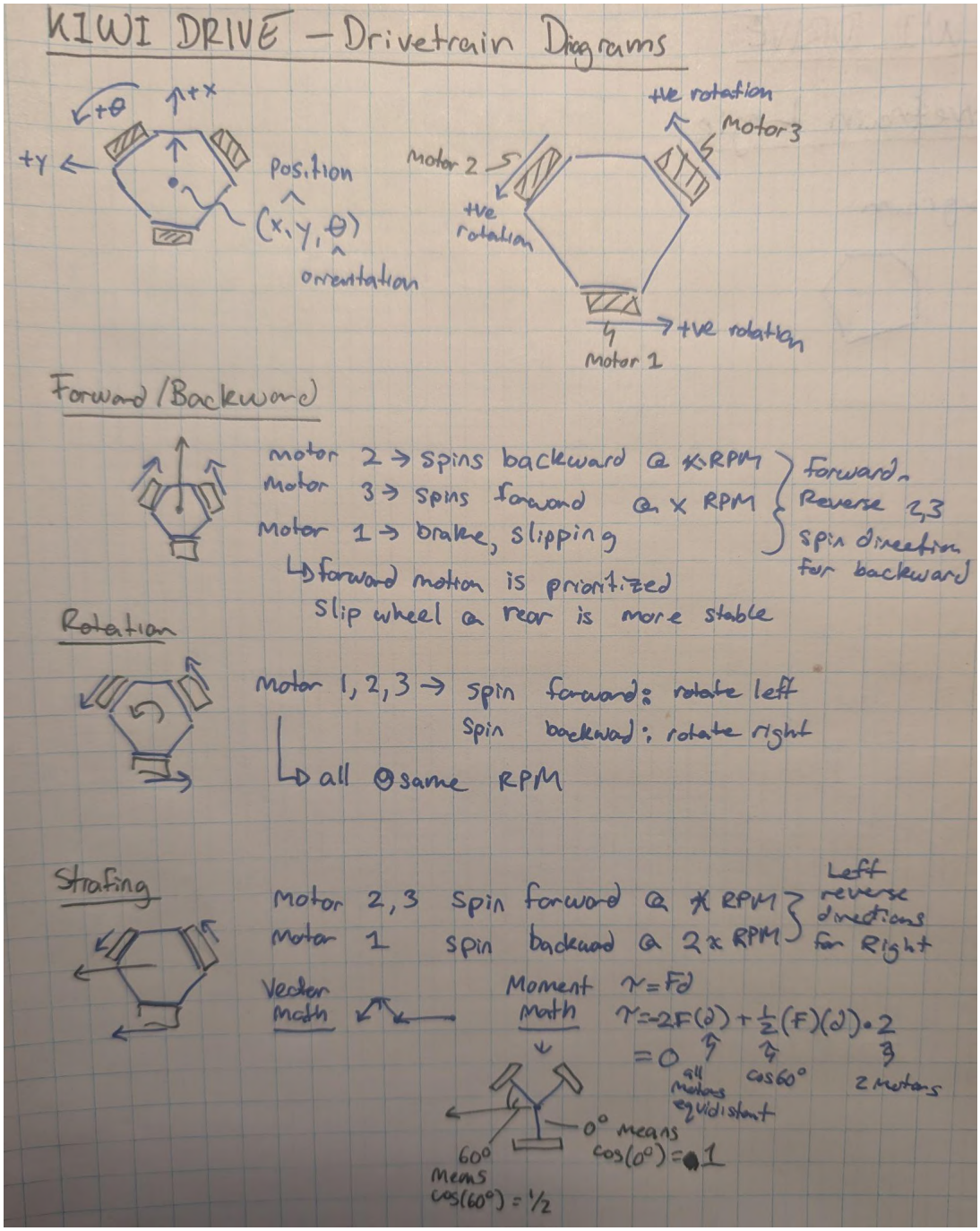


Figure 19: Kiwi Drive Movement Diagrams

D.2 Kinematics

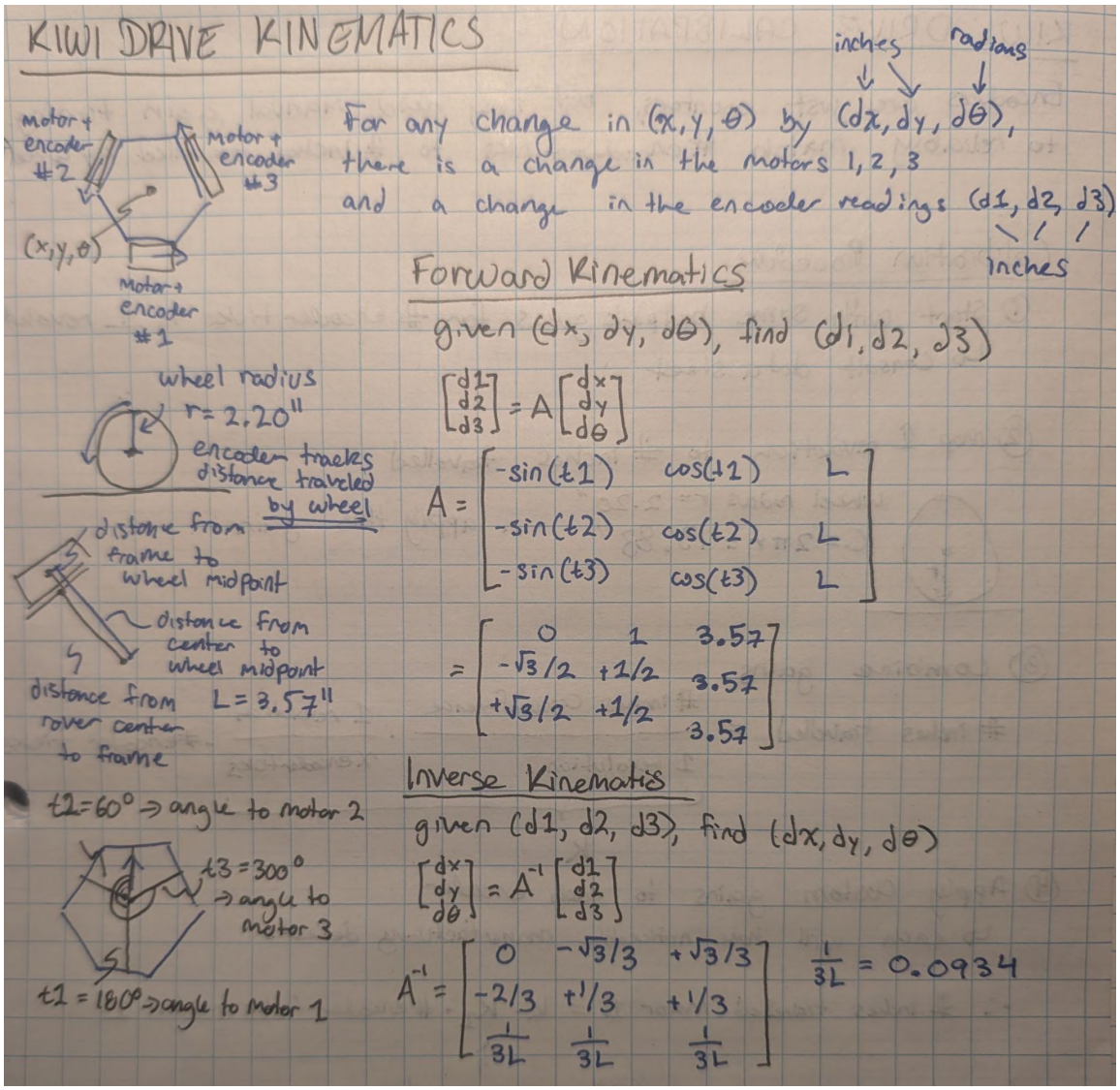
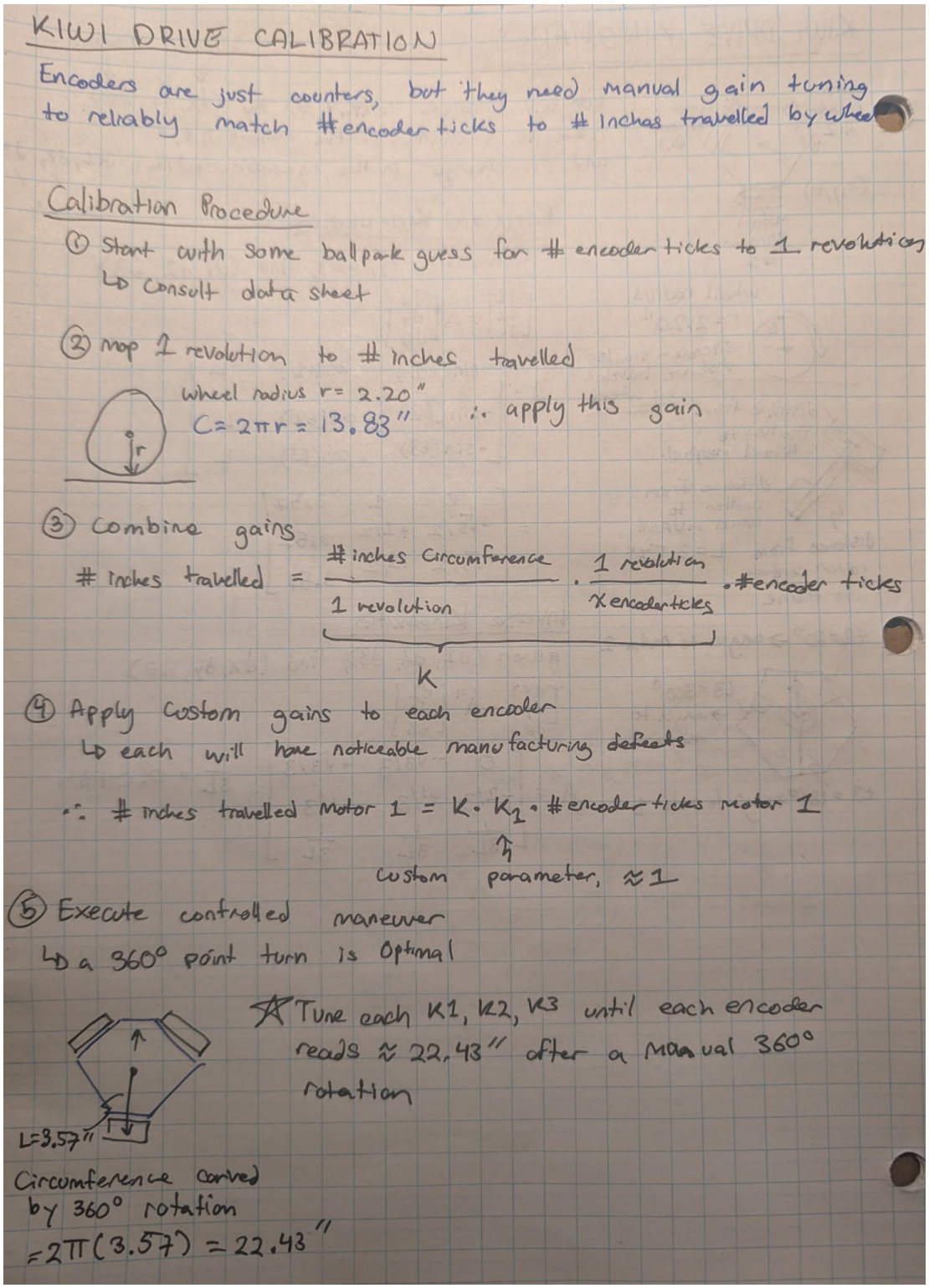


Figure 20: Kiwi Drive Forward and Inverse Kinematics

D.3 Encoder Calibration Procedure



KIWI DRIVE CALIBRATION

Encoders are just counters, but they need manual gain tuning to reliably match #encoder ticks to # inches travelled by wheel

Calibration Procedure

① Start with some ballpark guess for # encoder ticks to 1 revolution
↳ Consult data sheet

② map 1 revolution to # inches travelled

wheel radius r = 2.20"
C = 2πr = 13.83" ∴ apply this gain

③ Combine gains

$$\#\text{ inches travelled} = \underbrace{\frac{\#\text{ inches Circumference}}{1\text{ revolution}} \cdot \frac{1\text{ revolution}}{x\text{ encoder ticks}}}_{K} \cdot \#\text{encoder ticks}$$

④ Apply custom gains to each encoder
↳ each will have noticeable manufacturing defects

∴ # inches travelled motor 1 = $K \cdot K_1 \cdot$ #encoder ticks motor 1

↑ custom parameter, ≈ 1

⑤ Execute controlled maneuver
↳ a 360° point turn is optimal

★ Tune each K1, K2, K3 until each encoder reads ≈ 22.43" after a manual 360° rotation

L=3.57"

Circumference carved by 360° rotation
= 2π(3.57) = 22.43"

Figure 21: Kiwi Drive Encoder Calibration Procedure